

FOOTPILOT

Application web de gestion de club de football

Dossier technique - Projet Annuel

Mastère 1 - Année universitaire 2025/2026

Équipe projet

Alix Pratabuy

Yoann Goumarre

Rendu du rapport : 10 juillet 2026 - Soutenance : 22 juillet 2026

Table des matières

1 - Introduction et contexte	1
Présentation générale du projet	1
Origine du projet et problématique	1
Objectifs du projet.....	2
Périmètre et structure du document	2
2 - Analyse du besoin et cahier des charges	2
Démarche d'analyse	2
Acteurs et rôles	3
Besoins fonctionnels.....	3
Besoins techniques et non fonctionnels	4
Matrice des permissions	4
Écarts entre le cahier des charges initial et le produit final	5
3 - Planification du projet	6
Organisation temporelle et jalons.....	6
Méthodologie de travail	7
Répartition des tâches et organisation de l'équipe	7
Stratégie de gestion de versions	7
Gestion des risques	8
4 - Conception et architecture	8
Architecture générale.....	8
Choix technologiques et avantages.....	9
Modèle de données.....	10
Architecture de l'API REST	11
Architecture du frontend	12
Infrastructure : conteneurisation et environnements	12
5 - Réalisation technique	13
Vue d'ensemble du code	13
Authentification et cycle de vie des comptes	13
Gestion des membres et hiérarchie des permissions	14
Équipes, effectifs et historisation	14
Planning et événements.....	14
Feuille d'appel : la logique de snapshot	15
Résultats, buts et statistiques	16
Fil d'actualités et notifications	17
Messagerie interne.....	17
Monétisation : le modèle freemium Stripe	18

Gestion des images.....	19
Qualité et lisibilité du code.....	19
6 - Tests et validation	20
Stratégie de test	20
Tests unitaires backend (Jest + Supertest)	20
Tests unitaires frontend (Vitest + Testing Library)	21
Tests bout-en-bout (Playwright)	21
Chaîne d'intégration continue (CI)	21
Déploiement continu (CD).....	22
Analyse de sécurité et résultats	23
Gestion des anomalies	23
Validation fonctionnelle	24
7 - Conclusion et perspectives.....	24
Synthèse du projet	24
Leçons apprises	24
Limites identifiées	25
Perspectives (version 2)	25
8 - Annexes	27
Annexe A - Schéma de données Prisma	27
Annexe B - Pipeline CI/CD.....	29
Annexe C - Captures d'écran de l'application.....	30
Annexe D - Conteneurisation	36
Annexe E - Tests	38
Annexe F - Déploiement et production.....	39
Annexe G - Cahier des charges	40
Annexe H - Extraits de code commentés	40

NB : après ouverture du document dans Word, faire clic droit sur la table des matières → « Mettre à jour les champs » pour générer les numéros de page.

1 - Introduction et contexte

Présentation générale du projet

FootPilot est une application web de gestion de club de football, pensée « mobile first », développée dans le cadre du projet annuel de Master 1 à l'ESGI (année universitaire 2025/2026). Elle s'adresse aux clubs amateurs et semi-professionnels et centralise en un seul outil l'ensemble de la vie administrative et sportive d'un club : gestion des membres et de leurs rôles, organisation des catégories et des équipes, planification des matchs et des entraînements, suivi de l'assiduité via des feuilles d'appel, saisie des résultats et des statistiques individuelles et collectives, communication interne via un fil d'actualités et une messagerie instantanée.

L'application repose sur une architecture web moderne en trois tiers : un frontend **React 18** (Vite, TypeScript, Tailwind CSS), une API REST **Node.js/Express** en TypeScript adossée à l'ORM **Prisma**, et une base de données **PostgreSQL 16**. L'ensemble est conteneurisé avec **Docker**, intégré en continu via **GitHub Actions** et déployé automatiquement en production : le frontend sur **Vercel** et le backend (avec sa base de données) sur **Railway**.

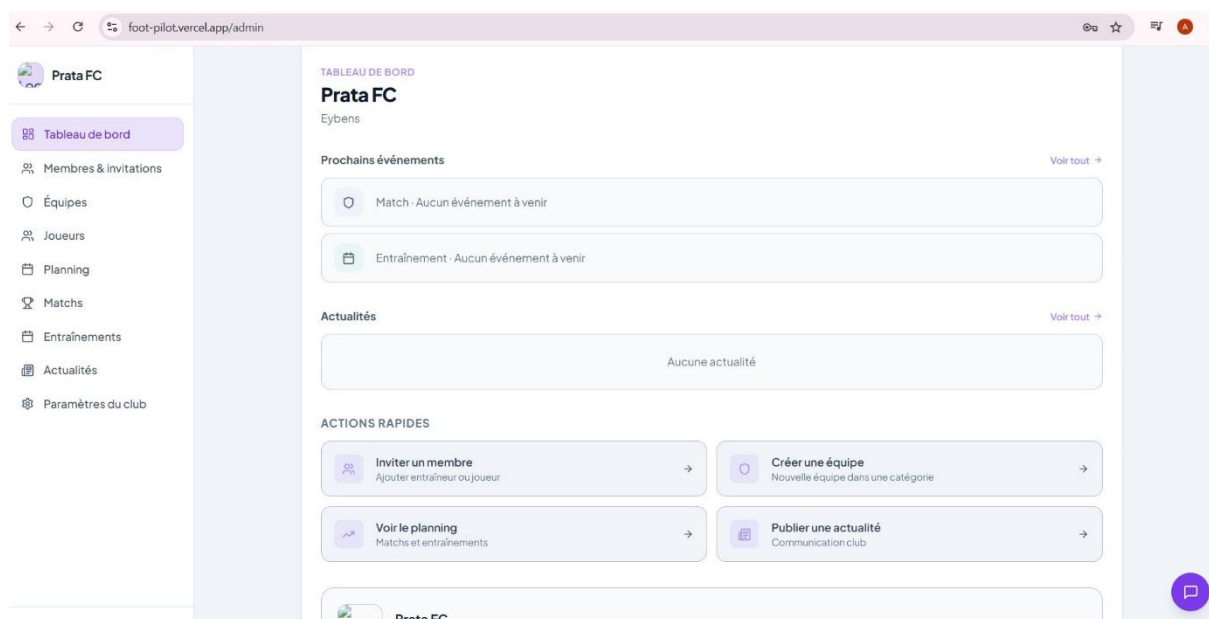


Figure 1 - Capture d'écran : page d'accueil (splash) de FootPilot en production

Origine du projet et problématique

Le constat de départ est simple et vécu par la plupart des clubs amateurs : la gestion d'un club de football repose aujourd'hui sur différents outils disparates et inadaptés. Les plannings circulent dans des groupes WhatsApp où l'information se perd, les effectifs et les cotisations sont dans des tableurs Excel maintenus par une seule personne, les feuilles de présence sont tenues sur papier quand elles le sont, et les statistiques des joueurs n'existent tout simplement pas. Les dirigeants bénévoles perdent un temps considérable en tâches administratives répétitives, les entraîneurs n'ont aucune visibilité historique sur l'assiduité ou les performances de leurs joueurs, et les joueurs eux-mêmes découvrent parfois un changement d'horaire trop tard.

La problématique adressée par le projet peut donc se formuler ainsi : **comment fournir aux clubs de football amateurs un outil unique, simple et accessible sur mobile, qui centralise la gestion administrative, la planification sportive et la communication interne, tout en restant économiquement viable pour des structures aux moyens limités ?**

Les solutions professionnelles existantes (logiciels fédéraux, plateformes SaaS destinées aux structures professionnelles) sont soit trop coûteuses, soit trop complexes pour un club animé par des bénévoles. FootPilot vise précisément ce créneau : un produit gratuit dans son usage de base, avec un modèle « freemium » (abonnement et achat unique via Stripe) pour les fonctionnalités avancées, posant ainsi les bases d'un futur produit SaaS rentable.

Objectifs du projet

Les objectifs fixés en début d'année, formalisés dans le cahier des charges (voir Annexe G), étaient les suivants :

- fournir un outil **tout-en-un** à destination des clubs de football amateurs et semi-professionnels ;
- **remplacer la gestion par tableurs** et outils disparates (WhatsApp, e-mails, papier) ;
- offrir une **visibilité claire** sur les plannings, les résultats et les statistiques, adaptée à chaque rôle (dirigeant, entraîneur, joueur) ;
- **faciliter la communication** entre dirigeants, entraîneurs et joueurs ;
- poser les bases d'un **futur produit SaaS** rentable (version 2).

À ces objectifs fonctionnels s'ajoutaient des objectifs pédagogiques : mener un projet complet en équipe sur une année (du cahier des charges à la mise en production), mettre en œuvre une méthodologie agile, industrialiser la qualité via une chaîne d'intégration et de déploiement continus (CI/CD) incluant tests automatisés et analyse de sécurité, et confronter nos choix techniques à des contraintes réelles d'hébergement et d'exploitation.

Périmètre et structure du document

Ce dossier technique retrace l'intégralité du projet. La section 2 analyse le besoin et synthétise le cahier des charges ; la section 3 décrit la planification et l'organisation de l'équipe ; la section 4 justifie les choix de conception et d'architecture ; la section 5 détaille la réalisation technique et la logique métier implémentée ; la section 6 présente la stratégie de tests, la chaîne CI/CD et la gestion des anomalies ; la section 7 dresse le bilan critique et les perspectives ; la section 8 référence les annexes (captures d'écran, extraits de code, documents complémentaires).

2 - Analyse du besoin et cahier des charges

Démarche d'analyse

L'analyse du besoin a été menée au premier semestre à partir de trois sources : l'expérience directe de membres de l'équipe ayant évolué en club amateur, des échanges informels avec des dirigeants et entraîneurs de clubs locaux, et une étude des solutions existantes sur le marché. Cette phase a abouti à la rédaction d'un cahier des charges structuré (versionné, environ 25 pages - voir Annexe G) comprenant le contexte, les objectifs, le périmètre, la matrice des permissions, les fonctionnalités

détaillées, le modèle de données cible, les user stories par acteur, l'architecture API, l'organisation projet et la roadmap en deux versions.

Le périmètre a volontairement été découpé en deux versions : une **v1** réalisable dans l'année (instance unique, fonctionnalités cœur) et une **v2** projetée (SaaS multi-tenant, application mobile, fonctions avancées). Ce découpage nous a servi de limite tout au long de l'année pour arbitrer les demandes d'évolution, même si, comme détaillé plus bas, certaines fonctionnalités initialement prévues en v2 (paiement Stripe, messagerie) ont finalement été intégrées à la v1 car le rythme de développement l'a permis.

Acteurs et rôles

L'application définit trois rôles hiérarchiques, chacun correspondant à un profil utilisateur réel du club. Le cahier des charges initial prévoyait un quatrième rôle ADMIN (super-administrateur de la plateforme, en préparation du SaaS v2). Il a été retiré du périmètre v1 lors de la conception détaillée, car il n'apportait aucune valeur à une instance mono-plateforme et complexifiait la matrice de permissions. C'est un exemple de simplification assumée par rapport au cahier des charges initial.

Rôle	Profil réel	Niveau d'accès
GESTIONNAIRE	Président ou responsable administratif du club (le créateur du club en est le « propriétaire »)	Club complet : membres, invitations, catégories, équipes, suppressions, abonnement
ENTRAINEUR	Responsable d'une ou plusieurs équipes	Ses équipes : joueurs, événements, feuilles d'appel, résultats, publications
JOUEUR	Membre du club	Lecture : son planning, ses statistiques, le fil d'actualités, le chat de son équipe

Tableau 1 - Les trois rôles applicatifs

Une subtilité supplémentaire a émergé pendant la réalisation : au sein du rôle GESTIONNAIRE, le **propriétaire** du club (celui qui l'a créé, champ `idOwner` sur le club) dispose de prérogatives exclusives : lui seul peut désactiver ou supprimer un autre gestionnaire. Cette hiérarchie implicite protège le club contre la prise de contrôle par un gestionnaire secondaire.

Besoins fonctionnels

Les besoins fonctionnels ont été formalisés en user stories regroupées par acteur (26 user stories au total dans le cahier des charges - US-GES-01 à 08, US-ENT-01 à 08, US-JOU-01 à 06, plus les stories d'administration). Ils se synthétisent en huit modules :

- **Authentification et comptes** : création de club publique (le créateur devient gestionnaire propriétaire), connexion par e-mail/mot de passe, invitation par lien à usage unique valable 7 jours, inscription alternative par code à 6 caractères, vérification d'adresse e-mail ;
- **Gestion du club** : fiche club (nom, ville, logo, description), catégories d'âge (U9 à Seniors), équipes rattachées à une catégorie avec niveau de championnat ;
- **Gestion des effectifs** : profils joueurs (poste, numéro de maillot, date de naissance, photo) et entraîneurs, affectation multiple joueur↔équipes (prêt, surclassement) avec historisation des périodes, ajout manuel de joueurs sans compte ;

- **Planning et événements** : matchs (adversaire, lieu géolocalisé, covoiturage, score, statut) et entraînements (durée, catégorie), vue calendrier filtrée par équipe et par rôle, contraintes d'intégrité temporelle (pas de création dans le passé, pas de modification d'un événement terminé) ;
- **Feuille d'appel et assiduité** : cinq statuts de présence (présent, absent justifié, absent non justifié, retard, blessé), note de performance sur 5, commentaire, buts marqués ; figement (« snapshot ») de la liste d'appel à la fin de l'événement ;
- **Résultats et statistiques** : saisie du score et des buts détaillés (buteur, passeur, minute, zone de tir, circonstance, CSC), statistiques par joueur (assiduité, buts, passes, note moyenne), par équipe (victoires/nuls/défaites, buts marqués/encaissés, winrate) et par club ;
- **Communication** : fil d'actualités paginé avec notification e-mail automatique à tous les membres, messagerie instantanée par salons (équipe, staff, direction) avec compteurs de non-lus ;
- **Monétisation** : modèle freemium : plan gratuit limité (3 équipes, 30 joueurs), abonnement mensuel à 4,99 € débloquent statistiques et messagerie, achat unique à 9,99 € levant les limites de création.

Besoins techniques et non fonctionnels

- **Mobile first** : l'interface doit être pensée d'abord pour smartphone (navigation basse sur mobile, barre latérale sur desktop), les utilisateurs finaux consultant l'application au bord du terrain ;
- **Sécurité** : authentification JWT, mots de passe hachés avec bcrypt (facteur de coût 12), validation systématique des entrées côté serveur, contrôle d'accès par rôle sur chaque route, rate limiting, secrets gérés exclusivement par variables d'environnement, HTTPS en production ;
- **Cloisonnement des données** : chaque requête est filtrée par le club de l'utilisateur (clubId porté par le jeton JWT) : un membre d'un club ne peut jamais voir les données d'un autre club ;
- **Internationalisation** : interface traduite en 7 langues (français, anglais, allemand, espagnol, italien, chinois, arabe), le français étant la langue de référence ;
- **Portabilité et reproductibilité** : l'ensemble de la stack doit se lancer en une commande (docker compose up --build) sur n'importe quel poste ;
- **Qualité industrialisée** : lint, vérification de typage, tests unitaires et bout-en-bout, audit de dépendances et analyse statique de sécurité exécutés automatiquement à chaque push ;
- **Déploiement continu** : toute fusion sur la branche principale validée par la CI doit être automatiquement mise en production.

Matrice des permissions

La matrice ci-dessous synthétise les permissions par rôle telles qu'implémentées (elle affine la matrice du cahier des charges initial). Elle est appliquée symétriquement côté frontend (gardes de routes) et côté backend (middlewares), le backend restant l'unique source d'autorité.

Fonctionnalité	GESTIONNAIRE	ENTRAINEUR	JOUEUR
Gérer le profil du club (fiche, logo, abonnement)	✓ (propriétaire ou non)	-	-

Fonctionnalité	GESTIONNAIRE	ENTRAINEUR	JOUEUR
Inviter des membres / créer des codes d'accès	✓	✓ (joueurs et entraîneurs)	-
Promouvoir un entraîneur en gestionnaire	✓	-	-
Désactiver / supprimer un membre	✓ (gestionnaires : propriétaire uniquement)	✓ (joueurs uniquement)	-
Gérer les catégories	✓	-	-
Créer / gérer des équipes	✓	✓ (ses équipes)	-
Créer / gérer des événements	✓	✓ (ses équipes)	-
Saisir la feuille d'appel	✓	✓ (ses équipes)	-
Saisir score et buts	✓	✓	-
Publier une actualité	✓	✓	-
Supprimer une actualité	✓	-	-
Consulter le planning	✓ (club entier)	✓ (ses équipes)	✓ (son équipe)
Consulter les statistiques	✓	✓	✓ (les siennes)
Messagerie - salon Direction	✓	-	-
Messagerie - salon Staff	✓	✓	-
Messagerie - salon Équipe	✓ (tous)	✓ (ses équipes)	✓ (son équipe)

Tableau 2 - Matrice des permissions implémentée

Écarts entre le cahier des charges initial et le produit final

Le cahier des charges est un document de cadrage, pas un contrat figé : plusieurs écarts assumés méritent d'être tracés, car ils illustrent les arbitrages faits en cours d'année.

Prévu au cahier des charges	Réalisé	Justification
Backend Kotlin + Spring Boot, frontend Angular	Backend Node.js/Express + TypeScript, frontend React	Pivot technologique décidé avant le développement : un seul langage (TypeScript) sur toute la stack, écosystème plus productif pour une équipe réduite (voir §4)
4 rôles (dont ADMIN plateforme)	3 rôles	Le rôle ADMIN n'a de sens qu'en SaaS multi-tenant (v2)
Refresh token + invalidation serveur	JWT simple 7 jours	Simplification assumée pour la v1 ; identifiée comme limite (§7)
Dashboard analytique Metabase	Non réalisé	Dépriorisé au profit du paiement Stripe et de la messagerie ; les statistiques applicatives couvrent le besoin utilisateur
Paiement Stripe (prévu v2)	Intégré en v1	Le modèle freemium a été implémenté en avance de phase (abonnement, achat unique, webhooks)
Messagerie / chat (prévu v2)	Intégré en v1	Salons par équipe/staff/direction, réservé aux clubs abonnés

Prévu au cahier des charges	Réalisé	Justification
GitLab CI + SonarQube	GitHub Actions + CodeQL + npm audit	Le dépôt étant hébergé sur GitHub, l'intégration native était plus simple et gratuite
Tests JUnit/Jasmine/Cypress	Jest/Supertest, Vitest/Testing Library, Playwright	Alignement sur la stack TypeScript retenue
Inscription uniquement par invitation	Invitation + codes d'accès + création de club publique	La création publique de club est indispensable pour acquérir des clubs ; les codes simplifient l'embarquement de vestiaires entiers

Tableau 3 - Écarts entre cahier des charges et réalisation

3 - Planification du projet

Organisation temporelle et jalons

Le projet s'est déroulé sur l'ensemble de l'année universitaire 2025/2026, en quatre grandes phases : cadrage, conception, réalisation, industrialisation. Les jalons officiels étaient le rendu du cahier des charges au premier semestre, le **rendu du présent dossier technique le 10 juillet 2026** et la **soutenance le 22 juillet 2026**.

Phase	Période	Contenu	Livrable / jalon
1. Cadrage	Octobre - décembre 2025	Étude du besoin, entretiens avec des clubs, analyse de l'existant, définition du périmètre v1/v2, rédaction du cahier des charges, user stories, modèle de données cible	Cahier des charges v1.0 (Annexe G)
2. Conception	Janvier - mars 2026	Maquettes Figma (16 écrans, design system violet), revue des choix technologiques et pivot vers la stack TypeScript, conception du schéma de base de données Prisma, architecture API, mise en place du dépôt GitHub et de l'environnement Docker	Maquettes validées, dépôt initialisé, docker-compose fonctionnel
3. Réalisation	Avril - mai 2026	Développement itératif : authentification et rôles, gestion club/catégories/équipes/joueurs, planning et événements, feuille d'appel, statistiques, actualités, i18n (7 langues), mode sombre, messagerie, intégration Stripe	Version 1 fonctionnelle complète (jalon atteint début mai 2026)
4. Industrialisation	Juin 2026	Chaîne CI/CD GitHub Actions (lint, typecheck, tests, sécurité), tests unitaires et E2E, analyse CodeQL et corrections, mise en production Vercel + Railway, migration e-mail vers Brevo, rate limiting	Application en production, pipeline complet vert
5. Restitution	Juillet 2026	Rédaction du dossier technique, préparation de la démonstration et de la soutenance	Rapport (10/07/2026), soutenance (22/07/2026)

Tableau 4 - Phases et jalons du projet

L'historique Git objective cette progression : premier commit le 26 avril 2026 au démarrage du développement, jalons internes versionnés (0.0.1 → 0.0.6 pour le cœur applicatif entre fin avril et début mai, incluant déjà les traductions, la feuille d'appel et Stripe), puis la série 0.1.x → 0.3.x en juin correspondant à la montée en puissance de la CI/CD (lint + tests, CodeQL, déploiement automatique).

Environ **60 commits** ont été poussés sur la branche principale, complétés par des branches de fonctionnalités.

Méthodologie de travail

En début de projet, nous avons défini une méthode agile inspirée de Scrum : itérations de deux semaines, point d'avancement asynchrone sur Discord, revue en fin d'itération avec re-priorisation du backlog, tenu sur GitHub (issues et tableau Projects), et fusions par pull request validées par la CI. Anis Grigah (Product Owner) était garant de la cohérence fonctionnelle avec le cahier des charges, Yoann Goumarre (Scrum Master) de l'animation de l'équipe et de la levée des blocages.

Il faut être franc : ce cadre n'a malheureusement pas résisté aux aléas de l'année. Le départ d'Irvine, puis les absences d'Anis liées à des problèmes personnels, ont progressivement vidé les itérations et les revues de leur substance. Dans les faits, nous avons avancé à tâtons, au fil de l'eau : coordination par Discord et WhatsApp, entraide ponctuelle, priorités décidées au coup par coup plutôt que selon le rituel prévu. Deux garde-fous ont néanmoins tenu toute l'année et beaucoup compensé : la traçabilité des issues GitHub et la CI bloquante avant toute fusion, qui a fortement limité les régressions (voir §6).

Répartition des tâches et organisation de l'équipe

L'équipe était composée de quatre membres : Alix Pratabuy (développeur), Yoann Goumarre (Scrum Master), Anis Grigah (Product Owner) et Irvine Fleith (lead développeur). Deux aléas ont bouleversé l'organisation : le départ d'Irvine de l'école en cours d'année, puis la disponibilité très réduite d'Anis. La répartition ci-dessous, redéfinie après le départ d'Irvine, n'a donc été que partiellement tenue : dans les faits, le périmètre a surtout été absorbé au fil de l'eau par les membres disponibles.

La répartition par domaines telle qu'elle avait été définie :

Membre	Rôle projet	Domaines principaux
Irvine Fleith	Lead développeur (départ en cours d'année)	Backend (API Express, Prisma), logique métier, chaîne CI/CD, tests backend
Yoann Goumarre	Scrum Master + développeur	Backend, conception de la logique métier (feuille d'appel, permissions, freemium), CI/CD et déploiement (Vercel/Railway), animation d'équipe
Anis Grigah	Product Owner + développeur	Frontend React (pages, composants UI, contextes, i18n, mode sombre), cohérence fonctionnelle avec le cahier des charges, tests E2E
Alix Pratabuy	développeur	Cadrage technique initial, participation au cahier des charges et aux maquettes

Tableau 5 - Répartition des rôles et des domaines

Malgré ce fonctionnement dégradé, les décisions de logique métier les plus engageantes (feuille d'appel, permissions, modèle freemium) ont été discutées collectivement avant implémentation dès que c'était possible : ces règles touchent à la fois le backend, le frontend et le modèle de données. C'est ce qui a permis de garder un produit cohérent malgré une organisation devenue largement informelle.

Stratégie de gestion de versions

La stratégie Git a évolué avec la maturité du projet, en trois temps :

- **Phase de construction** : le dépôt étant un monorepo (dossiers frontend/ et backend/), chaque membre travaillait sur sa propre branche de longue durée, ce qui permettait d'avancer en parallèle sur des périmètres disjoints sans conflits ;
- **Consolidation de la V1** : une fois le cœur applicatif stabilisé, les branches ont été fusionnées sur main, qui est devenue la branche de référence protégée par la CI ;
- **Phase de fonctionnalités** : à partir de là, chaque évolution est passée par une branche courte nommée NEW/<nom_de_la_fonctionnalité> (par exemple NEW/changement_front_back, NEW/modif_front), fusionnée sur main par pull request après validation de la CI et relecture.

Ce fonctionnement en deux temps (branches longues puis branches courtes) est un compromis pragmatique : les branches longues conviennent au démarrage où chacun construit un pan entier de l'application, mais deviennent dangereuses (conflits, divergences) dès que le code se stabilise. Le passage aux branches courtes avec CI obligatoire a fluidifié la fin de projet. Les messages de commit suivent l'esprit des Conventional Commits (fix(ci): ..., feat: ..., test: ...), et le tag [skip ci] est utilisé pour les modifications purement documentaires afin d'économiser des minutes de CI.

Gestion des risques

Risque identifié	Impact	Mesure prise
Départ d'un membre (Irvine)	Perte de 25 % de la capacité, savoir non partagé	Réorganisation par domaines, documentation systématique (README, commentaires), montée en compétence croisée
Stack initiale (Spring/Angular) inadaptée à la capacité de l'équipe	Vélocité insuffisante, projet non terminé	Pivot assumé vers une stack TypeScript unique avant le début du développement
Régressions lors des fusions sur main	Application cassée en production	CI bloquante (lint, typecheck, tests, sécurité) obligatoire avant fusion et avant déploiement
Dérive du périmètre (« feature creep »)	Jalons non tenus	Découpage v1/v2 dans le cahier des charges, arbitrages du PO
Fuite de secrets (JWT, Stripe, SMTP)	Compromission des données	Variables d'environnement uniquement, .env exclus de Git, secrets GitHub Actions chiffrés, analyse CodeQL des workflows
Indisponibilité des services gratuits (Vercel/Railway)	Démo impossible	Docker Compose reproduisant l'intégralité de la stack en local comme plan de secours

Tableau 6 - Registre des risques

4 - Conception et architecture

Architecture générale

FootPilot suit une architecture web classique en **trois tiers**, organisée en **monorepo** : un dossier frontend/ (application React servie sous forme de fichiers statiques), un dossier backend/ (API REST Express) et une base de données PostgreSQL. Le frontend ne contient aucune logique d'autorité : il consomme exclusivement l'API via HTTP/JSON, l'API étant seule responsable de la validation, des règles métier et du contrôle d'accès. Ce découplage strict permet d'héberger les deux parties sur des

plateformes différentes (Vercel pour le statique, Railway pour l'API) et ouvrirait la voie, en v2, à d'autres clients (application mobile) consommant la même API.

Le choix du monorepo (plutôt que deux dépôts séparés) s'est imposé pour un projet étudiant : une seule CI, des pull requests qui peuvent embarquer de façon atomique une évolution touchant le front et le back, et un README unique pour l'installation. Les deux packages restent néanmoins totalement indépendants (dépendances, build, déploiement).

Choix technologiques et avantages

Le cahier des charges initial prévoyait Kotlin/Spring Boot et Angular. Avant d'écrire la première ligne de code, nous avons réévalué ce choix à cause de deux réalités : le départ annoncé de notre lead développeur (le plus à l'aise sur la JVM) et la nécessité de livrer une application complète en quelques mois. Nous avons pivoté vers une stack **100 % TypeScript**, décision structurante qui s'est révélée payante.

Composant	Technologie retenue	Avantages qui ont motivé le choix
Langage	TypeScript (front et back)	Un seul langage sur toute la stack : montée en compétence mutualisée, types partagés conceptuellement, chaque membre peut intervenir partout. Typage statique = classe entière d'erreurs éliminée à la compilation
Frontend	React 18 + Vite 5	Écosystème le plus riche, composants réutilisables ; Vite offre un démarrage instantané et un build optimisé (tree-shaking, code splitting)
Styles	Tailwind CSS 3	Développement d'interface très rapide, design system cohérent (palette violette de la maquette), mode sombre natif via la classe dark, purge automatique du CSS inutilisé
Routage front	React Router 6	Routage déclaratif, gardes d'accès par composant (RequireAuth) avec listes de rôles
HTTP client	Axios	Intercepteurs centralisés : injection automatique du jeton, déconnexion globale sur réponse 401
Backend	Node.js 20 + Express 4	Framework minimaliste et maîtrisé, middlewares composables (auth, rôles, abonnement, limites), immense écosystème npm
ORM	Prisma 5	Schéma déclaratif unique (source de vérité), client entièrement typé généré automatiquement, migrations simples (db push en dev), protection par construction contre les injections SQL
Base de données	PostgreSQL 16	SGBD relationnel robuste et gratuit, adapté aux nombreuses relations du domaine (club → catégories → équipes → joueurs), types riches (enums, bytes)
Validation	Zod	Schémas de validation déclaratifs sur chaque route, erreurs structurées renvoyées au client, inférence de types TypeScript
Authentification	JWT (jsonwebtoken) + bcryptjs	API stateless (aucune session serveur), jeton portant userId/role/clubId, hachage coût 12
Paieement	Stripe (Checkout + webhooks)	Aucune donnée bancaire ne transite par nos serveurs, pages de paiement hébergées certifiées PCI, webhooks signés pour la synchronisation des statuts
E-mails	Brevo (API transactionnelle)	300 e-mails/jour gratuits, délivrabilité correcte, API simple ; retenu après essais SMTP/Ethereal puis Resend (voir §6)

Composant	Technologie retenue	Avantages qui ont motivé le choix
Images	Multer + Sharp	Upload en mémoire (5 Mo max, liste blanche MIME), redimensionnement et conversion WebP qualité 82 avant stockage
Conteneurisation	Docker + Docker Compose	Environnement reproductible en une commande, images multi-stage légères (Alpine), parité dev/prod
CI/CD	GitHub Actions + CodeQL	Intégration native au dépôt, gratuit pour le volume du projet, analyse de sécurité SAST incluse
Hébergement front	Vercel	CDN mondial, HTTPS automatique, rewrites SPA, déploiement en secondes via CLI
Hébergement back + BDD	Railway	Build à partir de notre Dockerfile, PostgreSQL managé attendant, déclenchement conditionné au succès de la CI (« Wait for CI »)

Tableau 7 - Choix technologiques et justifications

Deux alternatives sérieusement étudiées méritent mention. **Jenkins ou GitLab CI** pour l'intégration continue : écartés car Jenkins impose de maintenir un serveur dédié et GitLab CI une migration du dépôt, alors que GitHub Actions est natif là où vit déjà le code. **Nodemailer/SMTP** pour les e-mails : c'était le choix initial, remplacé en cours de projet par l'API Brevo après des problèmes concrets de configuration et de délivrabilité en production (détaillés en §6, gestion des anomalies).

Modèle de données

Le schéma de base de données, décrit dans backend/prisma/schema.prisma (source de vérité unique, reproduit en Annexe A, compte **18 modèles** et **8 énumérations**. Il traduit fidèlement le modèle conceptuel du cahier des charges, avec quelques évolutions issues de la réalisation.

Modèle	Rôle	Points de conception notables
Club	Entité racine : tout appartient à un club	Porte le propriétaire (idOwner), les champs de facturation Stripe (stripeCustomerId, subscriptionStatus, hasUnlockedLimits) et un drapeau isFounder (clubs pionniers exemptés de limites)
User	Compte de connexion, quel que soit le rôle	E-mail unique, rôle (enum), rattachement au club, drapeaux isActive / emailVerified / isManual ; suppression en cascade contrôlée applicativement
Joueur / Entraîneur	Profils sportifs	Toujours liés 1-1 à un User (userId unique) : les noms vivent uniquement sur User (pas de duplication) ; un joueur créé manuellement reçoit un User « stub » inactif (isManual)
Categorie / Equipe	Organisation sportive	Une équipe appartient à une catégorie et au club ; suppression du club en cascade
JoueurEquipe	Jonction joueur ↔ équipe	Historisée : dateDebut/dateFin (dateFin nulle = affectation active). Permet prêts et surclassements et fonde la logique de « snapshot » des feuilles d'appel
EntraîneurEquipe	Jonction entraîneur ↔ équipe	Clé primaire composée, base du contrôle d'accès « entraîneur de cette équipe »

Modèle	Rôle	Points de conception notables
Evenement	Table unique polymorphe match/entraînement	Discriminée par l'enum TypeEvenement ; champs spécifiques matchs (adversaire, scores, statut, places de covoiturage) ou entraînements (catégorie) nullables ; champ snapshotPris pour la feuille d'appel
Presence	Feuille d'appel	Clé composée (evenementId, joueurId) ; statut (5 valeurs), note /5, buts, commentaire
But	Détail des buts d'un match	Buteur, passeur optionnel, minute, zone de tir, circonstance, drapeau contre-son-camp
Actualite	Fil d'actualités	Auteur, club, ciblage équipe optionnel
ChatRoom / ChatMessage / ChatReadReceipt	Messagerie	Salons typés (EQUIPE, STAFF, DIRECTION) ; accusés de lecture par (userId, roomId) pour les compteurs de non-lus
Invitation / JoinCode / EmailVerificationToken	Cycle de vie des comptes	Jetons à expiration ; invitation à usage unique (usedAt), code d'accès multi-usage (usedCount), vérification e-mail 24 h
Image	Stockage binaire des images	Octets WebP directement en base - choix discuté en §7

Tableau 8 - Modèles principaux du schéma Prisma

Le choix le plus débattu a été la modélisation des événements. Le cahier des charges prévoyait un héritage à trois tables (Evenement parent, Match et Entraînement enfants). Nous avons retenu une **table unique discriminée** par une énumération : Prisma ne supporte pas nativement l'héritage de tables, et les champs spécifiques sont peu nombreux. En contrepartie, quelques colonnes sont nulles selon le type : compromis classique « single table inheritance » que la validation Zod (schémas distincts par type, union discriminée) rend sûr à l'écriture.

Autre décision structurante : la **dénormalisation zéro des identités**. Dans une première version du schéma, Joueur et Entraîneur portaient leurs propres prénom/nom, dupliqués avec User. Une refonte a supprimé cette duplication : le nom vit sur User, et les routes API « aplatissent » la réponse (joueur.user.firstName → joueur.firstName) pour ne pas complexifier le frontend. Les joueurs ajoutés manuellement (sans adresse e-mail réelle) reçoivent un compte User factice inactif, marqué isManual, avec une adresse interne générée aléatoirement. Cela garantit l'invariant « tout joueur a un User » et permet, plus tard, de rattacher un vrai compte.

Architecture de l'API REST

L'API expose une soixantaine d'endpoints répartis en **15 routeurs Express** (un par ressource : auth, gestionnaire, clubs, categories, equipes, joueurs, entraineurs, evenements, statistiques, actualites, images, join-codes, chat, billing, webhooks). Toutes les routes sont préfixées par /api et renvoient du JSON avec des messages d'erreur en français. Une route GET /api/health sert de sonde de vie aux hébergeurs et aux tests.

Chaque requête traverse une chaîne de middlewares dont l'ordre est significatif :

- **CORS** restreint à l'origine du frontend (variable CORS_ORIGIN) ;
- le webhook Stripe est monté **avant** le parseur JSON, car la vérification de signature exige le corps brut de la requête ;

- parseur JSON limité à 2 Mo ;
- **rate limiting global** : 300 requêtes par tranche de 15 minutes et par adresse IP sur tout /api (express-rate-limit), complété par un limiteur spécifique anti-abus sur l'inscription par code (5 tentatives/heure/IP, 3/15 min/e-mail) ;
- verifyToken : vérifie le JWT du header Authorization et attache { userId, role, clubId } à la requête ;
- requireRole(...) : renvoie 403 si le rôle du porteur n'est pas dans la liste autorisée ;
- requireSubscription / checkClubLimits : middlewares de monétisation (renvoient 402 « abonnement requis » ou 403 « limite atteinte ») ;
- gestionnaire d'erreurs final : toute exception non gérée est journalisée et masquée derrière un 500 générique (pas de fuite de pile en production).

Au-delà du contrôle par rôle, chaque route applique un **contrôle d'appartenance** : vérification que la ressource visée appartient bien au club du porteur du jeton, et, pour les entraîneurs, qu'ils encadrent bien l'équipe concernée (jonction EntraîneurEquipe). Ce double niveau - rôle puis périmètre - est le cœur du cloisonnement multi-club de l'application.

Architecture du frontend

Le frontend est une **SPA** (single page application) organisée autour de conventions strictes :

- src/App.tsx déclare la quarantaine de routes, chacune enveloppée dans un composant RequireAuth paramétré par la liste des rôles autorisés ; un utilisateur non connecté est renvoyé vers /login, un rôle non autorisé vers son tableau de bord ;
- src/api/client.ts centralise l'instance Axios : le jeton stocké en localStorage est injecté dans chaque requête, et toute réponse 401 purge la session et redirige vers la connexion ; src/api/*.ts fournit des fonctions typées par ressource ;
- cinq **contextes React** portent l'état global : AuthContext (utilisateur, jeton, revalidation au chargement via /auth/me), BillingContext (statut d'abonnement, utilisé par les bannières paywall), ChatContext (salons, compteurs de non-lus), ThemeContext (mode sombre persisté, classe dark sur la racine), l18nContext (langue courante, dictionnaires JSON) ;
- les pages sont séparées entre pages/admin/ (interface gestionnaire/entraîneur) et pages/dashboard/ (interface entraîneur/joueur), avec des composants UI réutilisables (Button, Input, Modal, Card, Badge, Dialog, EmptyState...) déclinés en variantes claires et sombres ;
- la navigation est adaptative : barre latérale sur desktop, barre de navigation basse (BottomNav) sur mobile - concrétisation du parti pris mobile first.

L'internationalisation couvre **7 langues** (fr, en, de, es, it, zh, ar : soit plus de 400 clés par langue), le français étant la langue de référence. Le sélecteur de langue est accessible depuis toutes les pages.

Infrastructure : conteneurisation et environnements

Le fichier docker-compose.yml (Annexe D) décrit quatre services : **db** (postgres:16-alpine, volume persistant, healthcheck pg_isready), **backend** (construit depuis backend/Dockerfile, démarré seulement quand la base est saine), **frontend** (construit depuis frontend/Dockerfile, exposé sur le port 3000) et **adminer** (interface web d'administration de la base pour le développement). Toutes les

valeurs sensibles sont lues depuis un fichier `.env` racine jamais commité, documenté par un `.env.example`.

Les deux Dockerfiles sont **multi-stage** pour minimiser les images finales. Côté backend : un étage de build installe les dépendances, génère le client Prisma et compile le TypeScript ; l'étage final ne conserve que les dépendances de production, le code compilé et les binaires natifs nécessaires (client Prisma, Sharp). Au démarrage, le conteneur applique le schéma à la base (`prisma db push`) puis lance le serveur - c'est ce même Dockerfile qui est exécuté par Railway en production. Côté frontend : un étage Node construit le bundle Vite (l'URL de l'API étant injectée en argument de build), puis un étage **nginx:alpine** sert les fichiers statiques avec compression gzip, repli SPA (`try_files ... /index.html`) et proxy `/api` vers le backend pour l'usage en Compose.

Trois environnements coexistent ainsi avec une parité maximale : le **développement local** (base en Docker, front et back en rechargement à chaud), la **stack Compose complète** (identique à la production, utilisée pour les démonstrations et comme plan de secours) et la **production** (Vercel + Railway, décrite en §6 avec la chaîne de déploiement).

5 - Réalisation technique

Vue d'ensemble du code

Le projet représente environ 15 000 lignes de TypeScript réparties entre le backend (15 routeurs, 2 middlewares transverses, 3 bibliothèques internes : Prisma singleton, e-mails, traitement d'images) et le frontend (26 pages, une vingtaine de composants réutilisables, 5 contextes, 15 modules d'API typés, 7 dictionnaires de traduction). Cette section détaille les fonctionnalités livrées et, surtout, la logique métier qui a demandé le plus de conception. Les extraits de code significatifs sont regroupés en Annexe (voir Annexe A pour le schéma, Annexe E pour les tests).

Authentification et cycle de vie des comptes

Trois portes d'entrée coexistent, couvrant tous les scénarios d'embarquement d'un club :

- **Création de club** (publique) : le formulaire crée, dans une transaction atomique, le club puis son premier utilisateur GESTIONNAIRE, désigné propriétaire (`idOwner`). Le compte est créé non vérifié : un jeton de vérification valable 24 h est envoyé par e-mail, et la connexion est refusée tant que l'adresse n'est pas confirmée ;
- **Invitation nominative** : un gestionnaire (ou un entraîneur, pour les joueurs) saisit e-mail, prénom, nom et rôle ; une invitation à jeton unique, valable 7 jours par défaut, est enregistrée et le lien `/register/<token>` est envoyé par e-mail (le lien reste copiable depuis l'interface si l'e-mail échoue). À l'inscription, l'invitation est consommée (`usedAt`) dans la même transaction que la création du compte - un lien ne peut jamais servir deux fois - et le profil métier correspondant (Joueur ou Entraîneur) est créé automatiquement ;
- **Code d'accès** : pour embarquer un vestiaire entier sans saisir vingt adresses, un code de 6 caractères est généré (alphabet sans caractères ambigus - pas de O/0/I/1 - tiré via `randomInt`, générateur cryptographique sans biais), avec rôle porté, durée de validité paramétrable (1 à 168 h) et compteur d'utilisations. La route publique d'inscription par code est protégée par un double rate limiting (IP et e-mail) ajouté suite à l'analyse de sécurité.

La connexion vérifie successivement l'existence du compte, son activation (`isActive`), le mot de passe (`bcrypt`, messages d'erreur volontairement identiques pour ne pas révéler l'existence d'un compte) et la vérification de l'adresse. Elle délivre alors un **JWT signé valable 7 jours** portant `userId`, `role` et `clubId` - ces trois informations suffisent à tout le contrôle d'accès sans requête supplémentaire. Côté client, une réponse 401 quelle qu'elle soit purge la session et ramène à l'écran de connexion.

Gestion des membres et hiérarchie des permissions

La gestion des membres illustre la finesse des règles métier implémentées. Un entraîneur peut inviter, désactiver ou supprimer uniquement des joueurs. Un gestionnaire peut agir sur joueurs et entraîneurs, et promouvoir un entraîneur au rang de gestionnaire (seule promotion autorisée - pas de rétrogradation). Seul le **propriétaire** du club peut désactiver ou supprimer un autre gestionnaire : personne ne peut supprimer son propre compte. La suppression d'un membre est une transaction qui retire d'abord les profils métier (Joueur/Entraîneur) puis le compte, afin de ne jamais laisser de profil orphelin.

Équipes, effectifs et historisation

Les équipes appartiennent à une catégorie d'âge et sont encadrées par un ou plusieurs entraîneurs. Le créateur d'une équipe en devient automatiquement entraîneur (y compris un gestionnaire, pour lequel un profil entraîneur est créé à la volée). Cette règle garantit qu'aucune équipe n'est orpheline. Les règles d'affectation des entraîneurs sont asymétriques et ont demandé réflexion : un entraîneur peut s'auto-affecter uniquement à une équipe qui n'a encore aucun entraîneur (sinon, un entraîneur en place doit l'ajouter), tandis qu'un gestionnaire peut s'auto-affecter librement.

L'affectation joueur ↔ équipe n'est jamais supprimée : retirer un joueur d'une équipe pose une **date de fin** sur l'enregistrement de jonction. Cette historisation, invisible pour l'utilisateur, est le fondement de la cohérence temporelle de l'application : elle permet de savoir qui appartenait à quelle équipe à n'importe quelle date passée, propriété indispensable à la feuille d'appel décrite ci-dessous.

Planning et événements

Matches et entraînements partagent le même socle (équipe, date/heure, durée de 15 min à 8 h, lieu avec coordonnées GPS optionnelles pour l'affichage cartographique, description) et sont validés par des schémas Zod distincts réunis en union discriminée. Les règles temporelles sont strictes : impossible de créer un événement dans le passé, impossible de modifier ou supprimer un événement terminé (la fin étant calculée comme date + durée). Un entraîneur ne peut créer d'événement que pour ses propres équipes.

Le planning affiché dépend du rôle : le gestionnaire voit tout le club, l'entraîneur ses équipes, et le joueur applique une règle plus subtile. Pour le **futur**, il voit les événements de ses équipes actuelles ; pour le **passé**, il voit les événements où il figure dans la feuille d'appel, même s'il a changé d'équipe depuis. Un joueur transféré conserve ainsi son historique sans « hériter » du passé de sa nouvelle équipe.

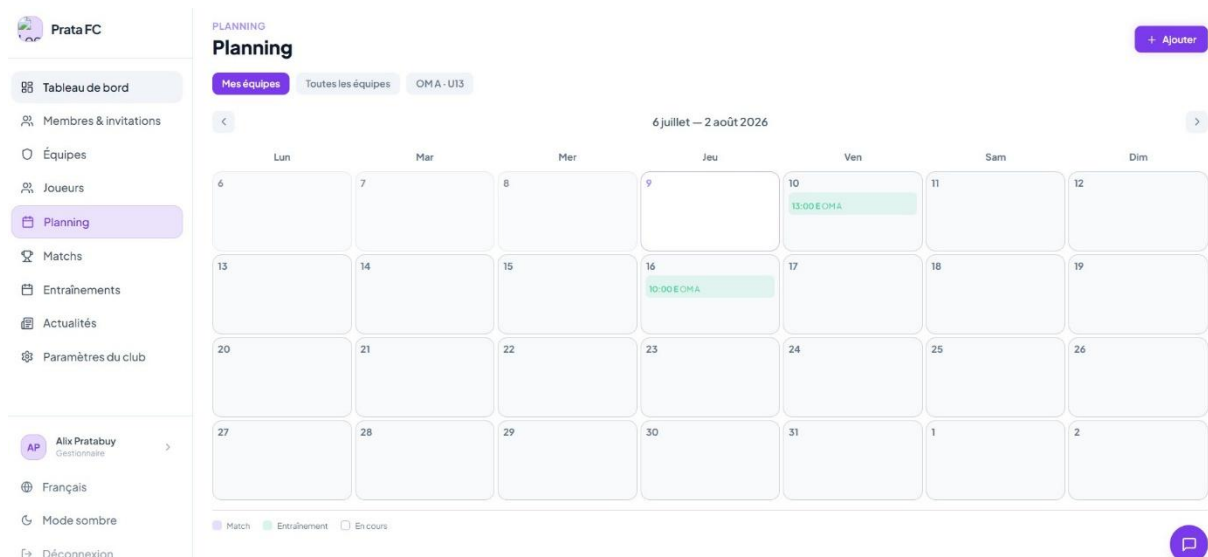


Figure 2 - Capture d'écran : vue planning (calendrier hebdomadaire) côté gestionnaire

Feuille d'appel : la logique de snapshot

La feuille d'appel est la pièce de logique métier la plus délicate du projet. Le problème : l'effectif d'une équipe évolue en permanence (arrivées, départs, transferts), mais une feuille d'appel doit refléter l'effectif **au moment de l'événement**, pas l'effectif actuel. Sans précaution, ajouter un joueur en septembre le ferait apparaître rétroactivement absent à tous les entraînements de la saison passée, faussant toutes les statistiques d'assiduité.

La solution implémentée distingue trois cas à l'ouverture d'une feuille d'appel :

- **Cas A - événement à venir ou en cours** : la liste est dynamique, construite à partir des affectations actives à l'instant T (date de début passée, pas de date de fin). Les statuts déjà saisis sont fusionnés par jointure, ce qui permet un pointage en direct pendant la séance ;
- **Cas B - événement terminé, première ouverture** : la liste est **figée** (« snapshot »). Une transaction reconstruit l'effectif tel qu'il était à la fin de l'événement grâce à l'historisation (affectations ayant commencé avant la fin de l'événement et non terminées à cette date), purge les éventuelles présences de joueurs qui avaient quitté l'équipe avant l'événement, crée les présences manquantes avec des valeurs par défaut (présent, note 3/5) et marque l'événement snapshotPris ;
- **Cas C - snapshot déjà pris** : la liste enregistrée est renvoyée telle quelle. Les saisies ultérieures ne peuvent que modifier les joueurs déjà présents dans le snapshot. Toute tentative d'y introduire un joueur étranger est silencieusement filtrée.

Ce mécanisme garantit que les feuilles d'appel passées sont immuables dans leur composition, tout en restant corrigibles dans leurs statuts (un entraîneur peut requalifier une absence en absence justifiée après coup). Les statistiques d'assiduité héritent automatiquement de cette exactitude historique.

Match

×

Match

Terminé

OMA - U13

ven. 12 juin, 16:00

2h

vs Prata FC

Pas qualif en Idc mdr

Score

Modifier le score

Score non renseigné

Liste d'appel

JOUEUR	STATUT	NOTE	BUTS
Joran Angela	Présent	★★★★☆	0
NanYuki Gro	Présent	★★★★☆	0

Valider l'appel

Cet événement est terminé.

Figure 3 - Capture d'écran : feuille d'appel d'un entraînement (statuts, notes, commentaires)

Résultats, buts et statistiques

Après un match, l'entraîneur saisit le score (ce qui bascule le statut à TERMINÉ) puis, s'il le souhaite, le détail de chaque but : buteur, passeur décisif optionnel, minute, zone de tir (tête, pied gauche, pied droit), circonstance (jeu ouvert, coup franc, penalty) et marquage contre-son-camp. Ces données alimentent trois niveaux de statistiques calculés à la volée :

- **par joueur** : assiduité détaillée par type d'événement (présences, absences justifiées ou non, retards, blessures), note moyenne sur 5, buts marqués, buts par match, passes décisives ;
- **par équipe** : matchs joués, victoires/nuls/défaites, buts marqués et encaissés, pourcentage de victoires ;
- **par club** : volumétrie globale (catégories, équipes, joueurs, membres, matchs, entraînements), affichée sur le tableau de bord du gestionnaire.

Le calcul à la volée (plutôt que des agrégats stockés) est un choix assumé pour la v1 : les volumes d'un club amateur (quelques centaines d'événements par saison) ne justifient pas la complexité d'une dénormalisation, et l'actualisation des chiffres est garantie par construction. Les statistiques joueur et équipe font partie du périmètre premium (voir plus bas), les statistiques club restant gratuites.

Match

×

Match

Terminé

OMA - U13

ven. 12 juin, 16:00

2h

vs Prata FC

Pas qualif en ldc mdr

Score

Modifier le score

1 – 0

Liste d'appel

JOUEUR	STATUT	NOTE	BUTS
Joran Angela	Présent	★★★★☆	0
NanYuki Gro	Présent	★★★★☆	0

Valider l'appel

Cet événement est terminé.

Figure 4 - Capture d'écran : saisie du score et des buts d'un match

Joran Angela

×

12/06/2026

ENTRAÎNEMENTS

● Présent	0
● En retard	0
● Abs. justifiée	0
● Blessé	0
● Absent	0

MATCHS

100%

1 séances

● Présent	1
● En retard	0
● Abs. justifiée	0
● Blessé	0
● Absent	0
★ Note moyenne	★★★★☆ 3.0
🎯 Buts marqués	0
> Buts / match	0.00

Figure 5 - Capture d'écran : page statistiques d'un joueur

Fil d'actualités et notifications

Gestionnaires et entraîneurs publient des actualités (titre, contenu, ciblage optionnel d'une équipe), consultables par tous les membres du club avec pagination (20 par page). À chaque publication, un e-mail est diffusé à tous les membres actifs du club via Brevo. L'envoi est asynchrone et non bloquant : un échec d'e-mail n'empêche jamais la publication, conformément au principe que la messagerie externe est un canal « best effort ». Seul un gestionnaire peut supprimer une actualité.

Messagerie interne

La messagerie (fonctionnalité premium) organise la communication en trois types de salons créés automatiquement à la demande : un salon par **équipe** (joueurs, entraîneurs de l'équipe et gestionnaires), un salon **staff** (entraîneurs et gestionnaires du club) et un salon **direction** (gestionnaires uniquement). Le contrôle d'accès à chaque salon réutilise la logique d'appartenance aux équipes. Les messages sont paginés par curseur temporel (40 par page, chargement de l'historique en remontant), et des accusés de lecture par utilisateur et par salon alimentent les compteurs de messages non lus affichés dans l'interface. Le rafraîchissement se fait par interrogation périodique (polling) : le passage aux WebSockets est identifié comme perspective (§7).

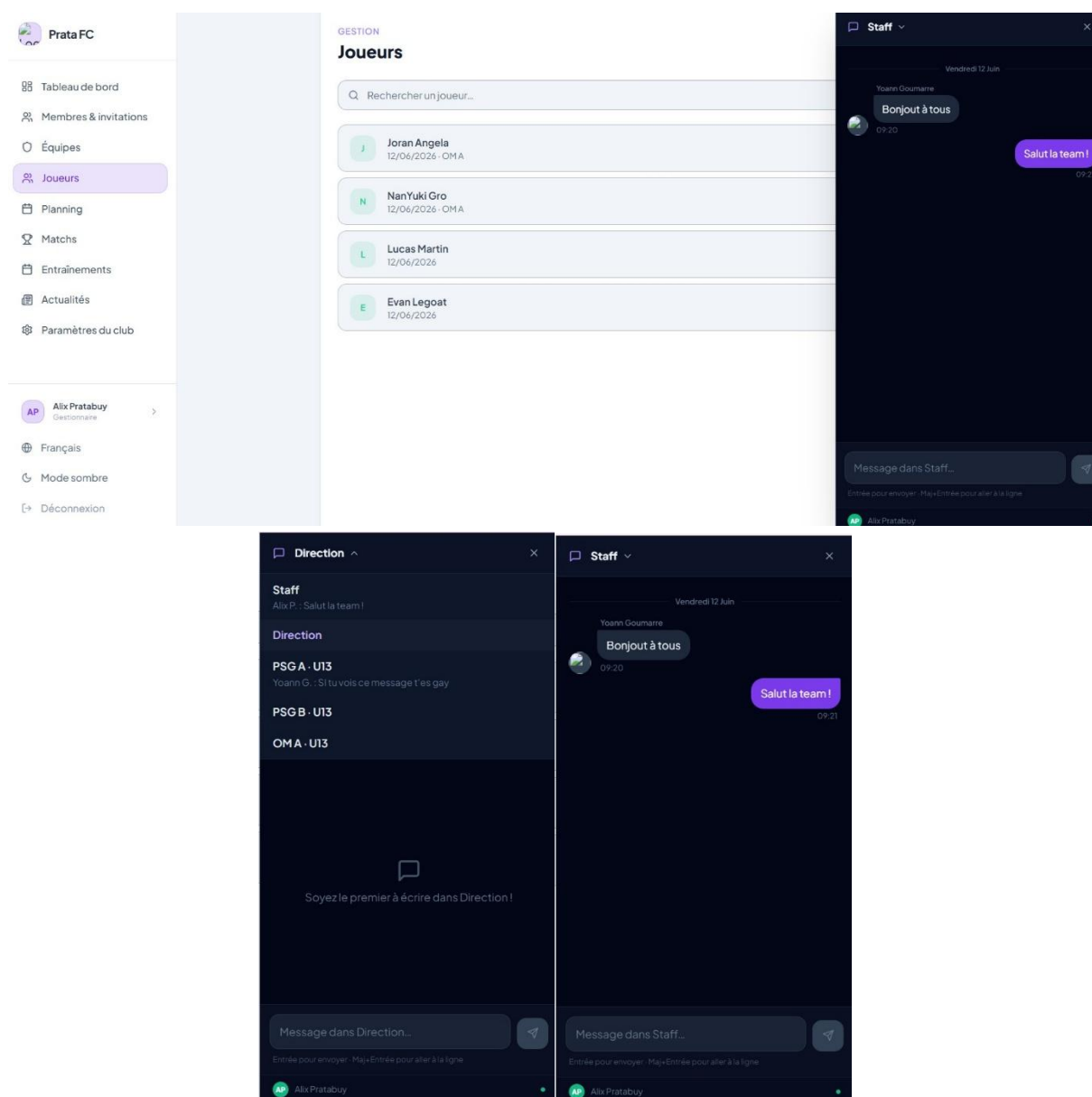


Figure 6 - Captures d'écran : messagerie (liste des salons et conversation)

Monétisation : le modèle freemium Stripe

Le modèle économique validé en cadrage est implémenté de bout en bout :

- **Plan gratuit** : toutes les fonctions de gestion, limité à 3 équipes et 30 joueurs par club ;
- **Abonnement Pro (4,99 €/mois)** : débloque les statistiques joueurs/équipes et la messagerie ;

- **Pack Illimité (9,99 €, achat unique)** : lève définitivement les limites de création ;
- **Clubs fondateurs** : un drapeau isFounder exonère de toutes les restrictions (early adopters).

Techniquement, le backend crée des sessions **Stripe Checkout** (mode abonnement ou paiement unique) en associant l'identifiant du club aux métadonnées, et un client Stripe est créé par club à la première transaction. La synchronisation des droits repose exclusivement sur les **webhooks signés** : `checkout.session.completed` active l'abonnement ou débloque les limites selon le mode, `customer.subscription.updated/deleted` répercute les changements d'état (actif, impayé, résilié). La signature de chaque webhook est vérifiée sur le corps brut de la requête, d'où le montage de cette route avant le parseur JSON. Un portail de facturation Stripe permet au gestionnaire de gérer son abonnement sans que nous ayons à développer ces écrans.

Côté application, deux middlewares font respecter le modèle : `requireSubscription` (renvoie 402 si le club n'est ni abonné ni fondateur, utilisé sur les statistiques et le chat) et `checkClubLimits` (renvoie 403 avec un code exploitable si le plafond d'équipes ou de joueurs est atteint, utilisé sur les créations). Le frontend interprète ces codes pour afficher des bannières d'incitation (« paywall ») plutôt que des erreurs brutes.

Figure 7 - Captures d'écran : page abonnement du club et bannière paywall sur une fonctionnalité premium

Gestion des images

Logos de clubs et photos de profil suivent le même pipeline : upload en mémoire (Multer, 5 Mo maximum, liste blanche de types MIME), redimensionnement à 400 px maximum et conversion **WebP qualité 82** (Sharp), puis stockage des octets en base de données et service via une route publique avec en-tête de cache immuable d'un an (les images étant adressées par identifiant unique, elles ne changent jamais). Ce choix « base de données » évite de dépendre d'un stockage objet externe en v1 ; ses limites sont discutées en §7.

Qualité et lisibilité du code

Plusieurs conventions maintiennent la lisibilité d'un code écrit à plusieurs mains : TypeScript en mode strict et zéro any toléré par ESLint des deux côtés ; validation Zod systématique en tête de chaque route d'écriture ; messages d'erreur utilisateur exclusivement en français ; helpers factorisés (normalisation des objets joueur, contrôles d'accès entraîneur/équipe) ; client Prisma singleton pour éviter les fuites de connexions en rechargement à chaud ; alias d'import @/ côté frontend ; règle documentée dans le guide du dépôt imposant que toute nouvelle clé de traduction ne soit ajoutée qu'au fichier français (les autres langues étant gelées en fin de projet pour éviter les divergences). Le README du dépôt (installation pas à pas, référence complète des routes de l'API, procédure d'activation des e-mails) complète cette documentation.

6 - Tests et validation

Stratégie de test

La stratégie suit la **pyramide des tests** : une base de tests unitaires rapides et déterministes, une couche de tests bout-en-bout (E2E) validant les parcours critiques dans un vrai navigateur, le tout complété par des vérifications statiques (lint, typage) et des analyses de sécurité automatisées. Le principe directeur : **aucun test ne dépend d'une infrastructure externe** (base de données, service e-mail, Stripe). Toutes les dépendances sont simulées (« mockées »), ce qui rend les tests exécutables en quelques secondes sur n'importe quelle machine et dans la CI.

Niveau	Outils	Cible	Isolation
Analyse statique	ESLint + tsc --noEmit (front et back)	Style, bonnes pratiques React/TS, typage strict	-
Tests unitaires backend	Jest + ts-jest + Supertest	Routes API (validation, logique métier, codes HTTP) et middlewares	Client Prisma mocké (jest.mock) : aucune base requise
Tests unitaires frontend	Vitest + Testing Library (jsdom)	Composants UI (rendu, états désactivé/chargement, variantes) et utilitaires	DOM simulé, aucune API appelée
Tests E2E	Playwright (Chromium)	Parcours réels : connexion, redirections par rôle, protection des routes, navigation	API entièrement mockée par interception réseau (page.route)
Sécurité	npm audit + CodeQL	Vulnérabilités des dépendances (CVE) et du code source (SAST)	-

Tableau 9 - Niveaux de test et outillage

Tests unitaires backend (Jest + Supertest)

Les tests backend s'appuient sur une décision d'architecture prise spécifiquement pour la testabilité : l'application Express est exportée par un module app.ts distinct du point d'entrée index.ts (qui ne fait que l'écouter sur un port). Supertest peut ainsi adresser des requêtes HTTP à l'application **sans démarrer de serveur**. Le client Prisma est remplacé par un mock : chaque test décrit exactement ce que la « base » doit répondre, ce qui permet de tester les embranchements de la logique métier un par un.

Trois suites couvrent les fondations critiques (15 cas de test) :

- **auth.test.ts** (9 cas) : validation Zod de la connexion (corps vide, e-mail malformé, mot de passe manquant → 400) puis logique métier complète (utilisateur inconnu, compte désactivé, mot de passe erroné, e-mail non vérifié → 401 ; identifiants corrects → 200 avec jeton signé, profil et statut de propriétaire) ;
- **middleware.test.ts** (4 cas) : le middleware verifyToken refuse l'absence de header, un schéma non Bearer et un jeton invalide (401), et attache correctement le payload décodé à la requête pour un jeton valide ;
- **health.test.ts** (2 cas) : sonde de vie et comportement 404 des routes inconnues.

Le choix de concentrer les tests unitaires sur l'authentification et le contrôle d'accès est délibéré : ce sont les composants dont la défaillance aurait l'impact le plus grave (accès non autorisé aux données d'un club), et ceux que tous les autres endpoints présupposent. La couverture des autres routes est identifiée comme axe d'amélioration (§7).

Tests unitaires frontend (Vitest + Testing Library)

Côté frontend, Vitest (moteur de test natif de l'écosystème Vite, configuration partagée avec le build) exécute les tests dans un DOM simulé (jsdom). Les tests ciblent les composants UI fondamentaux. Par exemple, pour le composant Button : rendu du libellé, désactivation via la propriété disabled, affichage du loader et désactivation en état de chargement, prise en compte des variantes de style (pleine largeur, danger), ainsi que les utilitaires purs (composition de classes CSS). Ces tests utilisent Testing Library, qui interroge le DOM comme le ferait un utilisateur (par rôle et libellé accessibles), rendant les tests robustes aux refontes internes des composants.

Tests bout-en-bout (Playwright)

Les tests E2E lancent l'application réelle (serveur Vite démarré automatiquement par Playwright) dans un navigateur Chromium et pilotent l'interface comme un utilisateur. L'API est **entièrement simulée par interception réseau** : chaque appel `/api/**` est intercepté et reçoit une réponse contrôlée. Ce choix rend les tests indépendants du backend et de toute base de données, indispensable pour la CI, tout en validant l'intégralité de la chaîne frontend : rendu, routage, contexte d'authentification, intercepteurs Axios, gardes de rôles.

Dix scénarios couvrent les parcours critiques :

- **Authentification** : affichage du formulaire ; connexion d'un gestionnaire → redirection vers `/admin` ; connexion d'un joueur → redirection vers `/dashboard` ; identifiants incorrects → message d'erreur affiché et maintien sur la page de connexion ;
- **Protection des routes** : accès à `/dashboard` sans jeton → redirection immédiate vers `/login` ;
- **Tableaux de bord** : affichage du nom de l'utilisateur connecté (joueur et gestionnaire) et du nom du club dans la barre latérale ;
- **Navigation** : liens de la barre latérale vers les pages membres et équipes.

En cas d'échec dans la CI, le rapport HTML de Playwright (avec traces et captures) est publié comme artefact téléchargeable pendant 7 jours, ce qui permet de diagnostiquer un échec survenu sur la machine distante sans le reproduire localement.

Chaîne d'intégration continue (CI)

Le pipeline, défini dans `.github/workflows/ci.yml` (reproduit en Annexe B), se déclenche à chaque push et chaque pull request vers main. Il est structuré en **cinq jobs** dont l'ordonnancement maximise le retour rapide : un job de vérifications statiques bloque tout le reste, puis trois jobs indépendants s'exécutent **en parallèle**, et le déploiement n'a lieu que si tous sont verts.

Job	Dépend de	Contenu	Rôle
1. Lint & TypeScript	-	npm ci puis ESLint et tsc --noEmit sur les deux packages	Barrière rapide (~1 min) : inutile de tester du code qui ne compile pas
2. Tests unitaires	Job 1	Vitest (frontend) puis Jest (backend) - compilation TypeScript en mémoire, secrets factices injectés	Détection des régressions fonctionnelles
3. Analyse sécurité	Job 1	npm audit (niveau high bloquant) sur les deux packages + CodeQL double passe : code applicatif TS/JS (requêtes security-extended) et fichiers de workflow eux-mêmes	Vulnérabilités de dépendances (CVE) et du code (SAST), résultats publiés dans l'onglet Security de GitHub
4. Tests E2E	Job 1	Installation de Chromium puis exécution Playwright ; rapport publié en artefact si échec	Validation des parcours utilisateur réels
5. Déploiement	Jobs 2 + 3 + 4	Build Vite via la CLI Vercel (variable VITE_API_URL injectée depuis les secrets) puis déploiement du bundle pré-construit ; Railway redéploie le backend en parallèle	Mise en production automatique - uniquement sur push direct sur main, jamais sur pull request

Tableau 10 - Les cinq jobs du pipeline CI/CD

Quelques choix d'ingénierie du pipeline méritent explication. **Parallélisation** : tests, sécurité et E2E étant indépendants, leur exécution simultanée réduit la durée totale du pipeline de moitié environ par rapport à un enchaînement séquentiel. **Machines éphémères** : chaque job démarre sur une machine vierge. Les dépendances sont réinstallées à chaque job (`npm ci`, versions exactes du lockfile), avec un cache npm indexé sur les lockfiles pour ne pas retélécharger les paquets. **Aucun build applicatif dans les jobs de test** : Vitest et Jest compilent le TypeScript en mémoire, et CodeQL analyse les sources directement ; le seul vrai build est celui du job de déploiement.

Déploiement continu (CD)

La mise en production est double et entièrement automatisée. Côté **frontend**, la CI construit le bundle avec la CLI Vercel : `vercel pull` récupère la configuration du projet, `vercel build --prod` compile le bundle en y figeant l'URL du backend de production (particularité de Vite : les variables d'environnement sont injectées au build, pas à l'exécution - d'où le choix de construire dans la CI, où la variable vit dans les secrets GitHub, plutôt que sur le dashboard Vercel, pour conserver une source de configuration unique), puis `vercel deploy --prebuilt` publie le dossier pré-construit sans recompilation. Côté **backend**, Railway surveille la branche main avec l'option « Wait for CI » : dès que le pipeline est vert, il reconstruit l'image depuis notre Dockerfile (compilation TypeScript, génération du client Prisma), applique le schéma à la base PostgreSQL managée et redémarre le service.

Quatre secrets GitHub chiffrés portent toute la configuration sensible du déploiement (jeton et identifiants Vercel, URL de l'API de production) ; les secrets d'exécution du backend (base de données,

JWT, Stripe, Brevo) vivent exclusivement dans les variables d'environnement Railway. Aucun secret n'apparaît dans le code ni dans l'historique Git.

Analyse de sécurité et résultats

L'analyse de sécurité automatisée a produit des résultats concrets. **CodeQL** a levé une alerte « missing rate limiting » sur les routes de statistiques : des endpoints effectuant des agrégations coûteuses étaient exposés sans limitation de débit, ouvrant un vecteur de déni de service. La correction a été double : un rate limiting ciblé sur la route concernée, puis, dans une itération ultérieure, un **limiteur global** de 300 requêtes par 15 minutes sur toute l'API, complété par le double limiteur spécifique (par IP et par e-mail) sur l'inscription publique par code. CodeQL analyse également les fichiers de workflow GitHub Actions eux-mêmes (injection de commandes, exposition de secrets), et **npm audit** en niveau bloquant « high » a imposé plusieurs montées de version de dépendances au fil du projet.

Au-delà de l'outillage, les mesures de sécurité applicatives sont récapitulées ici : hachage bcrypt coût 12, JWT signé porté en header (jamais en cookie tiers), validation Zod de toute entrée, contrôle rôle + appartenance sur chaque ressource, messages d'authentification non discriminants, vérification de signature des webhooks Stripe, jetons d'invitation/vérification à expiration et usage unique, codes d'accès générés par générateur cryptographique sans biais, CORS restreint, erreurs serveur masquées, images re-encodées systématiquement (neutralisant tout fichier malveillant déguisé en image).

Gestion des anomalies

Les anomalies étaient tracées comme issues GitHub et corrigées en priorité sur les développements (« stop the line »). L'historique Git en conserve la trace ; le tableau ci-dessous présente les plus instructives.

Anomalie	Cause	Correction
Échec du build Vercel dans la CI	La commande de build par défaut enchaînait tsc puis vite build : conflits de versions TypeScript et inclusion des fichiers de test dans la compilation de production	Création d'une commande de build dédiée (build:vercel), exclusion des fichiers de test du tsconfig de production, correction du PATH pour que la CLI Vercel trouve Vite
E-mails d'invitation non délivrés en production	Configuration SMTP inadaptée à l'hébergement (ports bloqués, délivrabilité médiocre)	Migration en deux temps : SMTP → Resend, puis Resend → API Brevo ; envois rendus asynchrones et non bloquants avec journalisation des échecs
Crash des tests backend en CI après l'intégration e-mail	Le client e-mail s'initialisait à l'import du module et exigeait une clé API absente en CI	Clé factice injectée dans l'environnement du job de test ; les tests mockent les appels externes
Test E2E « identifiants incorrects » instable	Le mock renvoyait 401, or l'intercepteur Axios global réagit à tout 401 en purgeant la session et en redirigeant - le message d'erreur disparaissait avant l'assertion	Le mock renvoie 400 (comportement réel de l'API pour un corps invalide) ; sélecteurs Playwright fiabilisés (rôles accessibles, href) ; un test de déconnexion structuellement instable a été retiré plutôt que maintenu erratique
Alerte CodeQL : absence de rate limiting	Endpoints de statistiques exposés sans limitation de débit	Rate limiting ciblé puis limiteur global 300 req/15 min (voir ci-dessus)

Anomalie	Cause	Correction
Minutes de CI consommées par un cron hebdomadaire	Une analyse CodeQL planifiée chaque samedi tournait sans changement de code	Suppression du déclencheur planifié : l'analyse à chaque push suffit pour un dépôt actif

Tableau 11 - Anomalies significatives et corrections

Validation fonctionnelle

En complément des tests automatisés, chaque fonctionnalité était validée manuellement contre ses critères d'acceptation (user stories du cahier des charges) sur un jeu de comptes de démonstration couvrant les trois rôles (gestionnaire propriétaire, entraîneur, joueurs). Les parcours transverses (création de club, invitation par e-mail réel, inscription par code, passage au plan payant via Stripe en mode test, webhooks) ont été rejoués de bout en bout sur l'environnement de production avant la démonstration. La stack Docker Compose locale, identique à la production, servait de banc de validation préalable.

7 - Conclusion et perspectives

Synthèse du projet

FootPilot est parti d'un constat de terrain - la gestion artisanale des clubs amateurs - et aboutit à une application web complète, **en production**, couvrant l'intégralité du périmètre v1 du cahier des charges et intégrant même des fonctionnalités initialement projetées en v2 (paiement Stripe, messagerie). Le projet valide la chaîne complète d'un produit logiciel : analyse du besoin, cahier des charges, maquettes, conception d'un modèle de données historisé, API REST sécurisée à trois niveaux de rôles, interface mobile first traduite en sept langues, pipeline CI/CD à cinq étages avec analyse de sécurité, et déploiement continu sur deux hébergeurs.

Sur le plan humain, l'année nous a confrontés à des situations réalistes : le départ d'un membre clé en cours de projet, un pivot technologique majeur assumé avant le développement, des anomalies de production (e-mails, builds) sans rapport avec ce qu'un environnement local laisse voir, et l'arbitrage permanent entre périmètre et échéances.

Leçons apprises

- **Un pivot technologique précoce vaut mieux qu'un enlisement tardif** : réévaluer la stack Spring/Angular en considération de notre capacité réelle, et pivoter avant la première ligne de code, a probablement sauvé le projet. Un cahier des charges doit cadrer le besoin, pas figer des choix d'outillage ;
- **La CI d'abord, les fonctionnalités ensuite** : les semaines qui ont suivi la mise en place du pipeline ont été les plus sereines du projet. Chaque fusion était validée automatiquement. Rétrospectivement, nous aurions dû installer la CI dès le premier commit plutôt qu'en phase d'industrialisation ;
- **Les outils de sécurité automatisés trouvent de vrais problèmes** : l'alerte CodeQL sur le rate limiting était fondée et a durci l'API. Il ne s'agit pas d'un exercice de conformité ;
- **La logique métier temporelle est ce qui se conçoit le plus difficilement** : le mécanisme de snapshot des feuilles d'appel a demandé plus de réflexion que des écrans entiers de CRUD. Les

cas limites (joueur transféré, événement en cours, saisie rétroactive) doivent être écrits noir sur blanc avant d'écrire le code ;

- **Tester en environnement réel tôt** : la majorité des anomalies (e-mails, build Vercel, variables d'environnement) n'existaient pas en local. Déployer tôt et souvent expose ces problèmes quand ils sont encore petits ;
- **Le risque humain se gère par le partage** : après le départ d'Irvine, la documentation et l'entraide croisée ont permis d'absorber le choc. Un savoir non partagé est une dette.

Limites identifiées

Nous portons un regard lucide sur les limites du produit livré :

- **Couverture de tests partielle** : les fondations (authentification, middlewares, parcours E2E critiques) sont testées, mais la logique métier la plus riche (snapshot d'appel, permissions d'équipes, statistiques) ne l'est pas encore automatiquement ; l'objectif de 70 % de couverture du cahier des charges n'est pas atteint ;
- **Session JWT simple** : pas de refresh token ni de révocation côté serveur. Un jeton volé reste valable jusqu'à 7 jours, le stockage en localStorage expose au risque XSS (mitigé par React et l'absence d'injection HTML, mais un cookie httpOnly serait plus robuste) ;
- **Schéma appliqué par db push** : l'absence de migrations versionnées (prisma migrate) est acceptable en v1 mais dangereuse à terme. Le déploiement applique le schéma avec une option qui accepte les pertes de données en cas de changement destructif ;
- **Messagerie en polling** : le rafraîchissement périodique génère des requêtes inutiles et une latence perceptible par rapport à des WebSockets ;
- **Images en base de données** : simple et cohérent en v1, mais fait grossir la base et monopolise le backend pour servir des octets statiques. Un stockage objet avec CDN s'imposera ;
- **Rate limiting en mémoire** : les compteurs sont perdus au redémarrage et ne se partagent pas entre plusieurs instances. Il faudrait un magasin partagé (Redis) pour passer à l'échelle ;
- **Reporting interne abandonné** : le dashboard Metabase prévu au cahier des charges n'a pas été réalisé ;
- **Traductions gelées** : seule la version française est activement maintenue ; les six autres langues datent de la campagne de traduction initiale.

Perspectives (version 2)

La roadmap v2 du cahier des charges reste pertinente et s'enrichit des enseignements de l'année. Par ordre de priorité :

- **Consolidation** : migrations Prisma versionnées, refresh tokens avec rotation et cookies httpOnly, couverture de tests de la logique métier (snapshot, permissions) avec tests d'intégration sur base réelle (Testcontainers), monitoring et alerting en production (Sentry, uptime) ;
- **Temps réel** : WebSockets pour la messagerie et les notifications (convocations, changements d'horaire), notifications push via une **PWA** installable, plus réaliste qu'une application native pour l'équipe ;

- **Produit** : module de covoiturage complet (inscription des accompagnateurs, places restantes), exports CSV/PDF (effectifs, feuilles de match), tableau de bord analytique interne (Metabase) pour le pilotage de la plateforme ;
- **SaaS** : facturation à l'échelle, stockage objet + CDN pour les médias, rate limiting distribué, et le rôle ADMIN plateforme qui retrouvera alors sa raison d'être. L'architecture actuelle, cloisonnement strict par club déjà en place, API stateless, conteneurisation, a été pensée pour que cette évolution soit une extension et non une réécriture.

En conclusion, FootPilot atteint son double objectif : livrer un produit réel, utilisable aujourd'hui par un club amateur depuis un simple navigateur, et nous avoir fait traverser (avec ses réussites et ses accidents) l'ensemble du cycle de vie d'un logiciel moderne, du besoin utilisateur jusqu'à la production surveillée par une chaîne d'intégration continue.

8 - Annexes

Les annexes regroupent les extraits de code et les captures d'écran référencés dans le corps du rapport.

Annexe A - Schéma de données Prisma

Le fichier backend/prisma/schema.prisma est la source de vérité unique du modèle de données (18 modèles, 8 énumérations). Les blocs les plus significatifs sont reproduits ci-dessous ; le fichier complet est disponible dans le dépôt. Référencé aux sections 4 et 5.

Extrait 1 - backend/prisma/schema.prisma (blocs principaux)

```
enum Role {
  GESTIONNAIRE
  ENTRAINEUR
  JOUEUR
}

enum TypeEvenement {
  MATCH
  ENTRAINEMENT
}

enum StatutPresence {
  PRESENT
  ABSENT_JUSTIFIE
  ABSENT_NON_JUSTIFIE
  RETARD
  BLESSE
}

model User {
  id          String      @id @default(cuid())
  email       String      @unique
  password    String
  firstName   String
  lastName    String
  role        Role        @default(JOUEUR)
  birthDate   DateTime?
  profilePic  String?
  phone       String?
  isActive    Boolean     @default(true)
  emailVerified Boolean    @default(true)
  isManual    Boolean     @default(false)
  clubId      String?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt
  // ... relations (club, profils joueur/entraîneur, invitations, messages)
}

model Club {
  id          String      @id @default(cuid())
  nom         String
  ville       String
  logoUrl     String?
  description  String?
  idOwner     String?
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  stripeCustomerId String? @unique @map("stripe_customer_id")
  subscriptionStatus String @default("free") @map("subscription_status")
  hasUnlockedLimits Boolean @default(false) @map("has_unlocked_limits")
  isFounder      Boolean @default(false) @map("is_founder")
  // ... relations (owner, users, categories, equipes, joueurs, chatRooms)
}
```

```

model Joueur {
    id          String      @id @default(cuid())
    userId      String      @unique
    clubId      String
    birthDate   DateTime
    poste       Poste?
    numeroMaillot Int?
    photoUrl    String?

    user User @relation(fields: [userId], references: [id])
    club Club @relation(fields: [clubId], references: [id], onDelete: Cascade)

    equipes    JoueurEquipe[]
    presences  Presence[]
    butsMarques But[] @relation("ButsMarques")
    butsPasses But[] @relation("ButsPasses")
}

model JoueurEquipe {
    id          String      @id @default(cuid())
    joueurId    String
    equipeId    String
    dateDebut   DateTime @default(now())
    dateFin     DateTime?

    joueur Joueur @relation(fields: [joueurId], references: [id], onDelete: Cascade)
    equipe Equipe @relation(fields: [equipeId], references: [id], onDelete: Cascade)

    @@index([joueurId, equipeId])
    @@index([equipeId, dateFin])
}

model Evenement {
    id          String      @id @default(cuid())
    type        TypeEvenement
    equipeId    String
    dateHeure   DateTime
    duree       Int          @default(120)
    annule      Boolean      @default(false)
    lieu        String?
    latitude    Float?
    longitude   Float?
    description  String?
    createdAt   DateTime     @default(now())

    // Champs spécifiques aux matchs (null pour entraînements)
    adversaire   String?
    scoreDom     Int?
    scoreExt     Int?
    statutMatch  StatutMatch?
    placesCovoiturage Int?

    // Champs spécifiques aux entraînements (null pour matchs)
    categorieId  String?

    // Snapshot : true dès que la liste d'appel a été figée (premier accès après début de
    l'événement)
    snapshotPris Boolean @default(false)

    equipe    Equipe @relation(fields: [equipeId], references: [id], onDelete: Cascade)
    categorie  Categorie? @relation(fields: [categorieId], references: [id])
    buts      But[]
    presences Presence[]
}

model Presence {
    evenementId String
    joueurId    String

```

```

statut      StatutPresence
note        Int?
buts        Int?
commentaire String?

evenement Evenement @relation(fields: [evenementId], references: [id], onDelete: Cascade)
joueur      Joueur   @relation(fields: [joueurId], references: [id], onDelete: Cascade)

@@id([evenementId, joueurId])
}

// Autres modèles (Categorie, Equipe, Entraîneur, EntraîneurEquipe, But, Actualite,
// ChatRoom, ChatMessage, ChatReadReceipt, Invitation, JoinCode, Image, ...) : voir le dépôt.

```

Annexe B - Pipeline CI/CD

Structure du pipeline `.github/workflows/ci.yml` (extraits : déclencheurs, permissions et ossature des cinq jobs ; le fichier complet, commenté, est disponible dans le dépôt). Référencé à la section 6.

Extrait 2 - `.github/workflows/ci.yml` (déclencheurs et ossature des cinq jobs)

```

name: CI/CD FootPilot

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

permissions:
  contents: read
  security-events: write
  actions: read

jobs:
  # -----
  # Job 1 : Lint + TypeScript check
  # -----
  lint-and-typecheck:
    name: Lint & TypeScript
    runs-on: ubuntu-latest

    # (8 étapes : checkout, Node 20 + cache npm, npm ci, puis ESLint et tsc --noEmit
    # sur les deux packages – barrière rapide avant tout le reste)

  # -----
  # Job 2 : Tests unitaires
  # -----
  test:
    name: Tests unitaires
    # Attend que le Job 1 soit vert avant de démarrer
    needs: lint-and-typecheck
    runs-on: ubuntu-latest
    # (checkout, Node 20, npm ci, puis Vitest frontend et Jest backend)
    # Exécute les tests backend avec Jest – JWT_SECRET et BREVO_API_KEY sont des valeurs
    # fictives suffisantes pour les tests qui mockent les appels externes
    - name: Tests backend (Jest)
      working-directory: backend
      run: npm test
    env:
      JWT_SECRET: test_secret_ci
      BREVO_API_KEY: brevo_placeholder

  # -----
  # Job 3 : Sécurité – npm audit + CodeQL
  # Tourne en parallèle du job test (tous deux

```



```

# attendent que lint-and-typecheck soit vert)
# -----
security:
  name: Analyse sécurité
  # Tourne en parallèle du Job 2 – tous deux attendent seulement le Job 1
  needs: lint-and-typecheck
  runs-on: ubuntu-latest
  # (checkout, Node 20, npm ci, npm audit bloquant niveau high sur les deux packages,
  # puis double passe CodeQL :)
  - name: Initialiser CodeQL (javascript-typescript)
    uses: github/codeql-action/init@v3
    with:
      languages: javascript-typescript
      queries: security-extended

  # Analyse le code source TypeScript/JavaScript à la recherche de failles
  # (injections, XSS, rate limiting manquant, etc.) et publie les résultats
  # dans Security → Code scanning alerts sur GitHub
  - name: Analyse CodeQL (javascript-typescript)
    uses: github/codeql-action/analyze@v3
    with:
      category: /language:javascript-typescript

# -----
# Job 4 : Tests E2E – Playwright + API mockée
# Tourne en parallèle de test et security
# (attend seulement lint-and-typecheck)
# -----
e2e:
  name: Tests E2E (Playwright)
  needs: lint-and-typecheck
  runs-on: ubuntu-latest
  # (checkout, Node 20, npm ci, installation de Chromium, npm run test:e2e)
  - name: Upload rapport Playwright
    if: failure()
    uses: actions/upload-artifact@v4
    with:
      name: playwright-report
      path: frontend/playwright-report/
      retention-days: 7

# -----
# Job 5 : Déploiement
# Frontend → Vercel
# Backend → Railway (auto via "Wait for CI")
# Ne s'exécute que sur push main, après que
# test ET security soient verts
# -----
deploy:
  name: Déploiement
  # Attend que les Jobs 2, 3 ET 4 soient verts – testé, sécurisé, et E2E validé
  needs: [test, security, e2e]
  # Ne se déclenche que sur push direct sur main, pas sur les pull requests
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  # (build Vite via la CLI Vercel puis déploiement – reproduit en Annexe F)

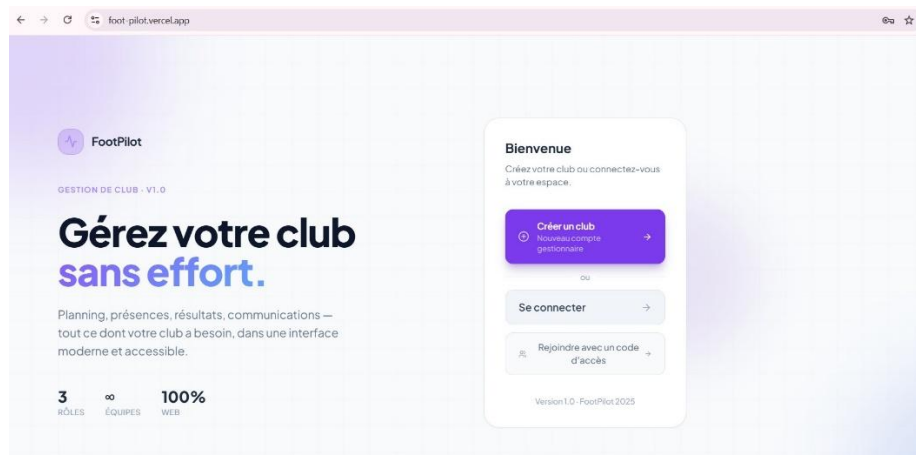
```

Annexe C - Captures d'écran de l'application

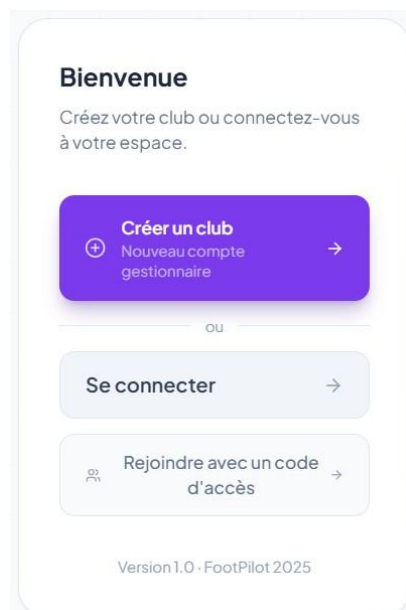
Parcours illustré de l'application en production :

- pages publiques : accueil (splash), connexion, création de club, inscription via invitation, inscription via code (/join), écran de vérification d'e-mail ;

- parcours gestionnaire : tableau de bord (statistiques du club), membres et invitations (avec la fenêtre d'invitation), équipes et détail d'équipe, joueurs, planning, création de match et d'entraînement, actualités, page club avec l'onglet abonnement ;
- parcours entraîneur : feuille d'appel (avant et après snapshot), saisie du score et des buts ;
- parcours joueur : tableau de bord, planning, statistiques personnelles ;
- transverse : messagerie (salons et conversation), bannière paywall, mode sombre, sélecteur de langue (interface en arabe ou en chinois pour illustrer).



Accueil public : présentation et points d'entrée (desktop)



Carte d'entrée : créer un club, se connecter, rejoindre avec un code



Connexion

Accédez à votre espace club

EMAIL

✉ alixprata@gmail.com

MOT DE PASSE

🔒 ••••••••

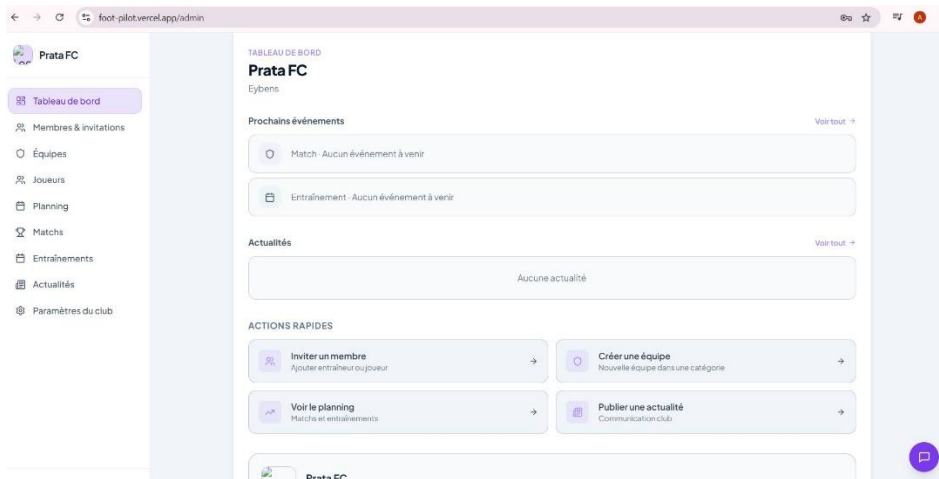
[Mot de passe oublié ?](#)

Se connecter →

🔑 Rejoindre avec un code d'accès

[← Retour à l'accueil](#)

Page de connexion



Prata FC

Tableau de bord

Membres & invitations

Équipes

Joueurs

Planning

Matches

Entraînements

Actualités

Paramètres du club

TABLEAU DE BORD

Prata FC

Eybens

Prochains événements

Match - Aucun événement à venir

Entraînement - Aucun événement à venir

Actualités

Aucune actualité

ACTIONS RAPIDES

Inviter un membre
Ajouter entraîneur ou joueur

Créer une équipe
Nouvelle équipe dans une catégorie

Voir le planning
Matches et entraînements

Publier une actualité
Communication club

Tableau de bord gestionnaire : prochains événements, actualités et actions rapides



Prata FC

Tableau de bord

Membres & invitations

Équipes

Joueurs

Planning

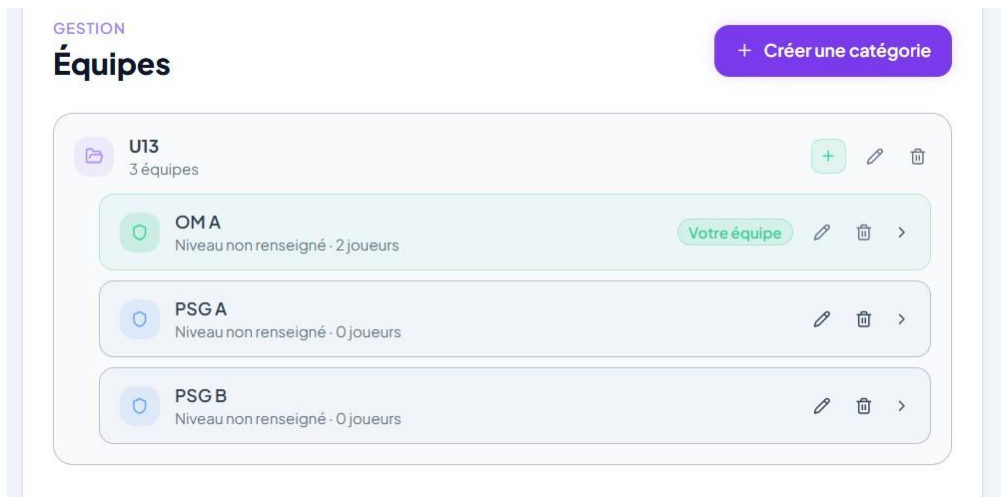
Matches

Entraînements

Actualités

Paramètres du club

Navigation latérale (desktop) de l'interface gestionnaire



Gestion des équipes, regroupées par catégorie d'âge

The screenshot shows the 'Nouvelle catégorie' (New category) form. It has a title bar with 'Nouvelle catégorie' and a close button. Below the title bar, there's a section 'NOM DE LA CATÉGORIE' with a text input field containing 'Ex : U13, Seniors, U17...'. At the bottom, there are two buttons: 'Annuler' (Cancel) and 'Créer' (Create).

Fenêtre de création d'une catégorie

The screenshot shows the 'Nouvel événement' (New event) form. It has a title bar with 'Nouvel événement' and a close button. Below the title bar, there are several fields: 'TYPE' (dropdown menu with 'Entraînement' selected), 'ÉQUIPE' (dropdown menu with 'OMA · U13' selected), 'DATE ET HEURE' (calendar icon and text input with '10/07/2026 13:00'), 'DURÉE' (dropdown menu with '2h' selected), 'LIEU (OPTIONNEL)' (text input with 'Rechercher une adresse...'), and 'DESCRIPTION (OPTIONNEL)' (text input with 'Informations complémentaires...'). At the bottom, there are two buttons: 'Annuler' (Cancel) and 'Créer' (Create).

Fenêtre de création d'un événement (entraînement)

Entraînements

+ Ajouter

Mes équipes

Toutes les équipes

OMA - U13

À VENIR

	Entraînement	À venir	OMA - U13	>
ven. 10 juil., 13:00				
	Entraînement	À venir	OMA - U13	>
jeu. 16 juil., 10:00				

Liste des entraînements à venir (filtres par équipe)

Entraînement

Entraînement

À venir

OMA - U13

ven. 10 juil., 13:00

2h

Modifier l'événement

Annuler l'événement

Supprimer

Détail d'un entraînement : modification, annulation, suppression

Nouvelle actualité

TITRE

Résultats du weekend

CONTENU

Bonjour à tous,

Les résultats du weekend sont disponible. Nous laissons les joueurs présents regarder leurs statistiques et nous en discuterons aux prochaines séances

Une notification email sera envoyée à tous les membres du club.

Annuler

Publier

Rédaction d'une actualité (notification e-mail aux membres du club)

Résultats du weekend

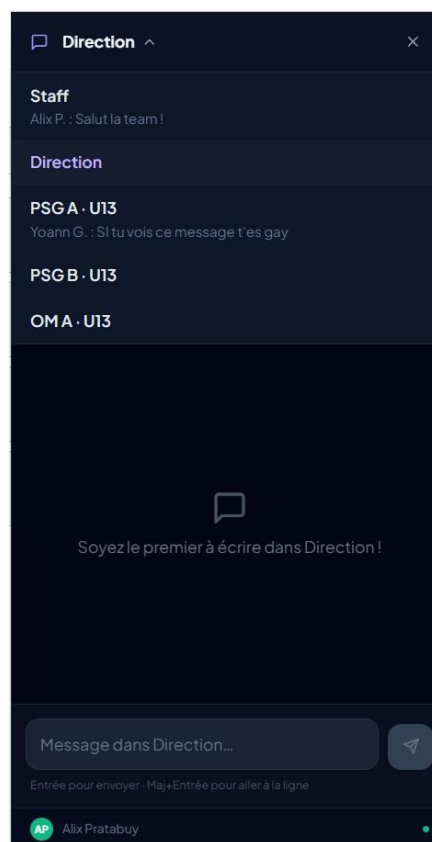


Bonjour à tous,

Les résultats du weekend sont disponible. Nous laissons les joueurs présents regarder leurs statistiques et nous en discuterons aux prochaines séances



Alix Pratabuy · il y a 0 min

Fil d'actualités du club*Messagerie : liste des salons (mode sombre)*



Messagerie : conversation du salon Staff (mode sombre)

Annexe D - Conteneurisation

Le fichier docker-compose.yml (quatre services) et les deux Dockerfiles multi-stage sont reproduits ci-dessous. Référencé à la section 4.

Extrait 3 - docker-compose.yml (les quatre services)

```
version: '3.9'

# Toutes les variables sensibles sont lues depuis le fichier .env à la racine.
# Copier .env.example → .env et renseigner les valeurs avant de lancer.

services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 5s
      retries: 10

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    restart: unless-stopped
```

```

ports:
  - "3001:3001"
environment:
  DATABASE_URL: postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
  JWT_SECRET: ${JWT_SECRET}
  PORT: 3001
  NODE_ENV: production
  APP_URL: ${APP_URL}
  CORS_ORIGIN: ${CORS_ORIGIN}
  # ... variables SMTP et Stripe, également lues depuis .env
depends_on:
  db:
    condition: service_healthy

frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
    args:
      VITE_API_URL: ${VITE_API_URL}
  restart: unless-stopped
  ports:
    - "3000:80"
  depends_on:
    - backend

adminer:
  image: adminer:latest
  restart: unless-stopped
  ports:
    - "8080:8080"
  environment:
    ADMINER_DEFAULT_SERVER: db
    ADMINER_DESIGN: pepa-linha
  depends_on:
    db:
      condition: service_healthy

volumes:
  postgres_data:

```

Extrait 4 - backend/Dockerfile (multi-stage : build TypeScript + Prisma, image finale allégée)

```

FROM node:20-alpine AS builder
RUN apk add --no-cache openssl
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY prisma ./prisma
RUN npx prisma generate
COPY . .
RUN npm run build

FROM node:20-alpine
RUN apk add --no-cache openssl
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev --ignore-scripts
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules/.prisma ./node_modules/.prisma
COPY --from=builder /app/node_modules/@prisma ./node_modules/@prisma
COPY --from=builder /app/node_modules/sharp ./node_modules/sharp
COPY --from=builder /app/node_modules/@img ./node_modules/@img
COPY prisma ./prisma

EXPOSE 3001
CMD ["sh", "-c", "npx prisma db push --accept-data-loss && node dist/src/index.js"]

```


Extrait 5 - frontend/Dockerfile (multi-stage : build Vite, service nginx)

```
FROM node:20-alpine AS builder
WORKDIR /app
ARG VITE_API_URL=http://localhost:3001
ENV VITE_API_URL=$VITE_API_URL
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Annexe E - Tests

Extraits représentatifs des suites de test : auth.test.ts (Jest + Supertest, client Prisma mocké) et auth.spec.ts (Playwright, API mockée par interception réseau). Référencé à la section 6.

Extrait 6 - backend/src/__tests__/auth.test.ts (Jest + Supertest, Prisma mocké)

```
// Mock Prisma pour ne pas toucher la vraie DB
jest.mock('../lib/prisma', () => ({
  prisma: {
    user: { findUnique: jest.fn() },
    club: { findUnique: jest.fn() },
  },
}));

const mockUser = prisma.user as unknown as { findUnique: jest.Mock };
const mockClub = prisma.club as unknown as { findUnique: jest.Mock };

beforeEach(() => {
  jest.clearAllMocks();
  process.env.JWT_SECRET = 'test_secret_ci';
});

describe('POST /api/auth/login - validation Zod', () => {
  it('retourne 400 si le body est vide', async () => {
    const res = await request(app).post('/api/auth/login').send({});
    expect(res.status).toBe(400);
    expect(res.body.message).toBe('Données invalides.');
```

```

const res = await request(app)
  .post('/api/auth/login')
  .send({ email: 'admin@club.fr', password: 'bonmdp123' });
expect(res.status).toBe(200);
expect(typeof res.body.token).toBe('string');
expect(res.body.user.email).toBe('admin@club.fr');
expect(res.body.user.isOwner).toBe(true);
});
});

```

Extrait 7 - frontend/e2e/auth.spec.ts (Playwright, API mockée par page.route)

```

import { test, expect } from '@playwright/test';
import { mockGestionnaire, mockJoueur, mockAdminRoutes, mockJoueurRoutes } from './helpers';

test.describe('Authentification', () => {
  // ... affichage du formulaire, connexion gestionnaire → /admin, connexion joueur → /dashboard
  test("identifiants incorrects affiche un message d'erreur", async ({ page }) => {
    await page.route('/api/auth/login', (route) =>
      route.fulfill({
        // 400 et non 401 – l'intercepteur Axios redirige vers /login sur 401,
        // ce qui efface le state React avant que l'erreur soit affichée
        status: 400,
        json: { message: 'Email ou mot de passe incorrect' },
      }),
    );

    await page.goto('/login');
    await page.getByPlaceholder('votre@email.com').fill('mauvais@test.com');
    await page.getByPlaceholder('.....').fill('mauvais');
    await page.locator('button[type="submit"]').click();

    await expect(page.getByText('Email ou mot de passe incorrect')).toBeVisible();
    await expect(page).toHaveURL('/login');
  });
});

```

Annexe F - Déploiement et production

Le déploiement est entièrement porté par le job deploy du pipeline, reproduit ci-dessous ; la configuration sensible tient dans quatre secrets GitHub chiffrés (VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID, VITE_API_URL). Référencé à la section 6.

Extrait 8 - job deploy de .github/workflows/ci.yml

```

# -----
# Job 5 : Déploiement
# Frontend → Vercel
# Backend → Railway (auto via "Wait for CI")
# Ne s'exécute que sur push main, après que
# test ET security soient verts
# -----
deploy:
  name: Déploiement
  # Attend que les Jobs 2, 3 ET 4 soient verts – testé, sécurisé, et E2E validé
  needs: [test, security, e2e]
  # Ne se déclenche que sur push direct sur main, pas sur les pull requests
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest

  steps:
    # Nouveau clone du code sur une machine virtuelle fraîche
    - name: Checkout code
      uses: actions/checkout@v4

```

```

# Installe Node.js 20 (sans cache ici – le deploy n'en a pas besoin)
- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'

# Installe la CLI Vercel globalement pour pouvoir utiliser les commandes vercel
- name: Install Vercel CLI
  run: npm install -g vercel@latest

# Installe les dépendances frontend – indispensable car vercel build va
# appeler vite qui se trouve dans node_modules/.bin
- name: Install frontend dependencies
  working-directory: frontend
  run: npm ci

# Télécharge la configuration du projet Vercel (settings, variables d'env de prod)
# Doit tourner depuis la racine du repo pour que Vercel trouve le bon rootDirectory
- name: Pull Vercel environment
  run: vercel pull --yes --environment=production --token=${{ secrets.VERCEL_TOKEN }}
  env:
    VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
    VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }

# Compile le frontend React/TypeScript en fichiers statiques optimisés –
# VITE_API_URL est injectée ici et figée dans le bundle JavaScript
- name: Build frontend
  run: |
    export PATH="$PWD/frontend/node_modules/.bin:$PATH"
    vercel build --prod --token=${{ secrets.VERCEL_TOKEN }}
  env:
    VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
    VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }
    VITE_API_URL: ${ secrets.VITE_API_URL }

# Envoie le dossier déjà compilé vers Vercel sans recompiler –
# le backend Railway se déploie automatiquement en parallèle via "Wait for CI"
- name: Deploy vers Vercel
  run: vercel deploy --prebuilt --prod --token=${{ secrets.VERCEL_TOKEN }}
  env:
    VERCEL_ORG_ID: ${ secrets.VERCEL_ORG_ID }
    VERCEL_PROJECT_ID: ${ secrets.VERCEL_PROJECT_ID }

```

Annexe G - Cahier des charges

Le cahier des charges v1.0 complet est fourni en document séparé (environ 25 pages : contexte, périmètre v1/v2, matrice des permissions, fonctionnalités détaillées, modèle de données cible, 26 user stories, architecture API, organisation projet, roadmap). Référencé aux sections 2 et 3.

Annexe H - Extraits de code commentés

Extraits illustrant la logique métier discutée en section 5 : la route GET /api/evenements/:id/appeal (les trois cas du snapshot), les middlewares verifyToken / requireRole / requireSubscription / checkClubLimits, le traitement des webhooks Stripe et l'intercepteur Axios du frontend.

Extrait 9 - GET /api/evenements/:id/appeal : les trois cas du snapshot (backend/src/routes/evenements.ts)

```

// — GET /api/evenements/:id/appeal —————

router.get('/:id/appeal', async (req, res) => {
  const ev = await prisma.evenement.findUnique({
    where: { id: req.params.id },
    select: { id: true, equipeId: true, snapshotPris: true, dateHeure: true, duree: true },
  });
});

```

```

if (!ev) return res.status(404).json({ message: 'Événement introuvable.' });

const eventEndDate = new Date(new Date(ev.dateHeure).getTime() + ev.duree * 60000);
const now = new Date();

// Cas C – snapshot déjà figé : retourner les présences enregistrées
if (ev.snapshotPris) {
  const presences = await prisma.presence.findMany({
    where: { evenementId: ev.id },
    include: { joueur: { select: joueurSelect } },
  });
  return res.json(presences.map((p) => ({ ...p, joueur: normalizeJoueur(p.joueur) })));
}

// Cas B – événement terminé, première ouverture : figer le snapshot
if (now >= eventEndDate) {
  const teamMembers = await prisma.joueurEquipe.findMany({
    where: {
      equipeId: ev.equipeId,
      dateDebut: { lte: eventEndDate },
      OR: [{ dateFin: null }, { dateFin: { gt: eventEndDate } }],
    },
    select: { joueurId: true },
  });

  const finalRosterIds = teamMembers.map((je) => je.joueurId);

  await prisma.$transaction([
    // Purge des présences fantômes (joueurs exclus de l'équipe avant la fin de l'événement)
    prisma.presence.deleteMany({
      where: { evenementId: ev.id, joueurId: { notIn: finalRosterIds } },
    }),
    ...teamMembers.map((je) =>
      prisma.presence.upsert({
        where: { evenementId_joueurId: { evenementId: ev.id, joueurId: je.joueurId } },
        update: {},
        create: { evenementId: ev.id, joueurId: je.joueurId, statut: 'PRESENT', note: 3 },
      })
    ),
    prisma.evenement.update({ where: { id: ev.id }, data: { snapshotPris: true } }),
  ]);

  const presences = await prisma.presence.findMany({
    where: { evenementId: ev.id },
    include: { joueur: { select: joueurSelect } },
  });
  return res.json(presences.map((p) => ({ ...p, joueur: normalizeJoueur(p.joueur) })));
}

// Cas A – événement en cours ou à venir : liste dynamique
const liveMembers = await prisma.joueurEquipe.findMany({
  where: {
    equipeId: ev.equipeId,
    dateDebut: { lte: now },
    OR: [{ dateFin: null }, { dateFin: { gt: now } }],
  },
  include: { joueur: { select: joueurSelect } },
});

const liveMemberIds = liveMembers.map((je) => je.joueurId);

// LEFT JOIN sur Presence pour récupérer les stats déjà saisies (live tracking)
const existingPresences = await prisma.presence.findMany({
  where: { evenementId: ev.id, joueurId: { in: liveMemberIds } },
});
const presenceMap = new Map(existingPresences.map((p) => [p.joueurId, p]));

return res.json(liveMembers.map((je) => {
  const existing = presenceMap.get(je.joueurId);
  return {

```

```

    evenementId: ev.id,
    joueurId: je.joueurId,
    statut: existing?.statut ?? ('PRESENT' as const),
    note: existing?.note ?? 3,
    buts: existing?.buts ?? null,
    commentaire: existing?.commentaire ?? null,
    joueur: normalizeJoueur(je.joueur),
  };
}));
});

```

Extrait 10 - verifyToken et requireRole (backend/src/middleware/auth.ts)

```

export function verifyToken(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ message: 'Token manquant.' });
  }

  const token = header.slice(7);
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET!) as JwtPayload;
    req.user = payload;
    next();
  } catch {
    return res.status(401).json({ message: 'Token invalide ou expiré.' });
  }
}

export function requireRole(...roles: Role[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user || !roles.includes(req.user.role)) {
      return res.status(403).json({ message: 'Accès interdit.' });
    }
    next();
  };
}

```

Extrait 11 - requireSubscription et checkClubLimits (backend/src/middleware/billing.ts)

```

const EQUIPES_LIMIT = 3;
const JOUEURS_LIMIT = 30;

export async function requireSubscription(req: Request, res: Response, next: NextFunction) {
  const clubId = req.user?.clubId;
  if (!clubId) return res.status(403).json({ message: 'Club requis.' });

  const club = await prisma.club.findUnique({
    where: { id: clubId },
    select: { subscriptionStatus: true, isFounder: true },
  });
  if (!club) return res.status(403).json({ message: 'Club introuvable.' });

  if (club.isFounder || club.subscriptionStatus === 'active') return next();

  return res.status(402).json({
    message: 'Abonnement requis pour accéder à cette fonctionnalité.',
    code: 'SUBSCRIPTION_REQUIRED',
  });
}

export function checkClubLimits(resource: 'equipes' | 'joueurs') {
  return async (req: Request, res: Response, next: NextFunction) => {
    const clubId = req.user?.clubId;
    if (!clubId) return res.status(403).json({ message: 'Club requis.' });

    const club = await prisma.club.findUnique({

```

```

    where: { id: clubId },
    select: {
      isFounder: true,
      hasUnlockedLimits: true,
      _count: { select: { equipes: true, joueurs: true } },
    },
  });
  if (!club) return res.status(403).json({ message: 'Club introuvable.' });

  if (club.isFounder || club.hasUnlockedLimits) return next();

  const count = resource === 'equipes' ? club._count.equipes : club._count.joueurs;
  const limit = resource === 'equipes' ? EQUIPES_LIMIT : JOUEURS_LIMIT;

  if (count >= limit) {
    return res.status(403).json({
      message:
        resource === 'equipes'
        ? `Limite atteinte : ${EQUIPES_LIMIT} équipes maximum. Passez au Pack Illimité.`
        : `Limite atteinte : ${JOUEURS_LIMIT} joueurs maximum. Passez au Pack Illimité.`,
      code: 'LIMITS_REACHED',
      limit,
      current: count,
    });
  }

  return next();
};
}

```

Extrait 12 - Webhook Stripe : vérification de signature et activation (backend/src/routes/webhooks.ts)

```

// POST /api/webhooks/stripe – corps brut requis (monté avant express.json dans index.ts)
router.post('/stripe', async (req, res) => {
  const sig = req.headers['stripe-signature'] as string;

  let event: ReturnType<typeof stripe.webhooks.constructEvent>;
  try {
    event = stripe.webhooks.constructEvent(
      req.body as Buffer,
      sig,
      process.env.STRIPE_WEBHOOK_SECRET ?? ''
    );
  } catch {
    return res.status(400).json({ message: 'Signature Stripe invalide.' });
  }

  if (event.type === 'checkout.session.completed') {
    const session = event.data.object as {
      mode: string;
      metadata?: Record<string, string> | null;
    };
    const clubId = session.metadata?.clubId;
    if (!clubId) return res.status(400).json({ message: 'clubId manquant dans les métadonnées.' });

    await prisma.$transaction(async (tx) => {
      if (session.mode === 'subscription') {
        await tx.club.update({ where: { id: clubId }, data: { subscriptionStatus: 'active' } });
      } else if (session.mode === 'payment') {
        await tx.club.update({ where: { id: clubId }, data: { hasUnlockedLimits: true } });
      }
    });
  }

  // ... customer.subscription.updated / deleted : mise à jour du subscriptionStatus
  // (active, past_due ou canceled) du club correspondant au customer Stripe

  return res.json({ received: true });
});

```

```
});
```

Extrait 13 - Intercepteur Axios : injection du jeton, déconnexion sur 401 (frontend/src/api/client.ts)

```
import axios from 'axios';

const client = axios.create({
  baseURL: `${import.meta.env.VITE_API_URL ?? ''}/api`,
  headers: { 'Content-Type': 'application/json' },
});

client.interceptors.request.use((config) => {
  const token = localStorage.getItem('fp_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

client.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('fp_token');
      localStorage.removeItem('fp_user');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default client;
```